

Assignment 2 2AMM10 2025-2026

Group: [Name Editing]

Member 1: [Chang Qin]

Member 2: [Annebel Hattingh-Haasbroek]

Member 3: [Alexandru Vasu]

```
import torch
import torch.nn as nn
import torch.nn.functional as F
from torch_geometric.loader import DataLoader
from torch.utils.data import Dataset
from torchvision import transforms

import copy
import json
from pathlib import Path
from PIL import Image
import matplotlib.pyplot as plt
from sklearn.metrics import precision_score, recall_score
import random
import numpy as np
import pandas as pd
from collections import defaultdict
from transformers import SwinModel
from tqdm import tqdm

## Global Variables ##
random.seed(42)
np.random.seed(42)
torch.manual_seed(42)
DATA_DIR = "./data"
DROWSY_DATASET_DIR = Path(
    DATA_DIR, "drowsy_graph_dataset"
) # Path to the drowsy dataset
OBJ_CLS_DATASET_DIR = Path(
    DATA_DIR, "bdd8cls_balanced_300_30"
) # Path to the object classification dataset
DEVICE = (
    torch.device("cuda")
    if torch.cuda.is_available()
    else (
        torch.device("mps")
        if torch.backends.mps.is_available()
```

```

        else torch.device("cpu")
    )
)

```

```

C:\Users\20223084\.conda\envs\dl_code\Lib\site-packages\requests\
__init__.py:86: RequestsDependencyWarning: Unable to find acceptable
character detection dependency (chardet or charset_normalizer).
warnings.warn(

```

Task 1

```

class DrowsyFaceGraphDataset(Dataset):
    def __init__(self, root, split="train"):
        self.root = Path(root)
        self.split = split

        data_path = self.root / f"{split}.pt"
        class_to_id_path = self.root / "class_to_id.json"
        id_to_class_path = self.root / "id_to_class.json"

        if not data_path.exists():
            raise FileNotFoundError(f"Could not find {data_path}")

        self.graphs = torch.load(data_path, weights_only=False)

        with open(class_to_id_path, "r") as f:
            self.class_to_id = json.load(f)

        with open(id_to_class_path, "r") as f:
            self.id_to_class = json.load(f)

    def __len__(self):
        return len(self.graphs)

    def __getitem__(self, idx):
        return self.graphs[idx]

train_dataset = DrowsyFaceGraphDataset(root=DROWSY_DATASET_DIR,
split="train")
test_dataset = DrowsyFaceGraphDataset(root=DROWSY_DATASET_DIR,
split="test")
class_to_id: dict = train_dataset.class_to_id
id_to_class: dict = train_dataset.id_to_class

##

# Visualization helper
def plot_face_graph(

```

```

    graph, ax=None, title=None, node_size=8, edge_alpha=0.20,
    edge_width=0.4
):
    """Plot one face graph using its 2D normalized landmark
    coordinates."""
    if ax is None:
        _, ax = plt.subplots(figsize=(4, 4))

    coords = graph.x.cpu().numpy() # Nodes of the Graph
    edge_index = graph.edge_index.cpu().numpy() # Edges of the Graph

    xy = coords[:, :2]

    seen = set()
    for i, j in edge_index.T:
        a, b = int(i), int(j)
        key = tuple(sorted((a, b)))
        if key in seen:
            continue
        seen.add(key)
        ax.plot(
            [xy[a, 0], xy[b, 0]],
            [xy[a, 1], xy[b, 1]],
            linewidth=edge_width,
            alpha=edge_alpha,
        )

    ax.scatter(xy[:, 0], xy[:, 1], s=node_size)
    ax.invert_yaxis()
    ax.set_aspect("equal")
    ax.axis("off")

    if title is not None:
        ax.set_title(title)

    return ax

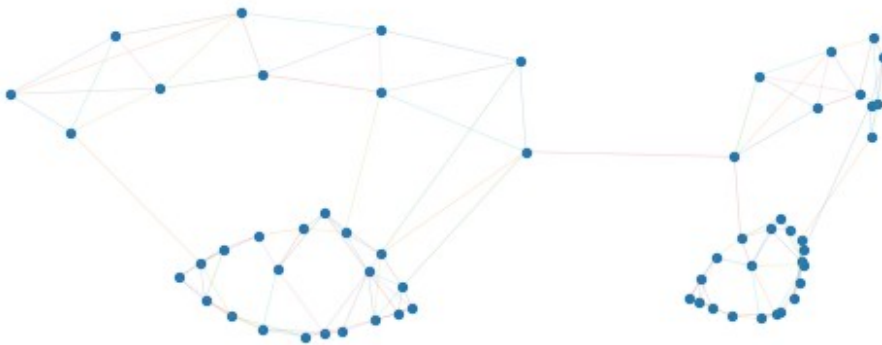
fig, ax = plt.subplots()
idx = random.randint(0, len(train_dataset) - 1)
graph = train_dataset[idx]
x, y = graph.x, graph.y
plot_face_graph(graph, ax)

# Understanding the samples
# 1) graph.x: (Num_nodes, 3) matrix. The coordinates (x1,x2,x3) of the
facial landmarks, normalized to [-1,1]. Those are the node features of
the graph.
# 2) graph.edge_index: (2, Num_edges) matrix. The edges of the graph,

```

where each column (i,j) represents an edge from node i to node j .
 # 3) graph.y: The label of the graph, where 0 means "alert" and 1 means "drowsy". This is the target variable we want to predict.
 # 4) You can safely ignore the other 2 attributes of the graph (source_index and eye_openness) for this assignment.
 # For completeness, eye_openness is a score in $[0,1]$ that indicates how open the eyes are, and source_index is an index corresponding to the sample index with respect to the original dataset before graph construction.

<Axes: >



```
# I set the seed again here so that rerunning Task 1 gives similar
results.
# The exact numbers can still change a little depending on the device.
seed = 42
random.seed(seed)
np.random.seed(seed)
torch.manual_seed(seed)
if torch.cuda.is_available():
    torch.cuda.manual_seed_all(seed)

# The PyG DataLoader combines several small face graphs into one
batch.
# Each node still keeps a graph id in data.batch, which is used later
for pooling.
device = DEVICE
train_loader = DataLoader(train_dataset, batch_size=32, shuffle=True)
test_loader = DataLoader(test_dataset, batch_size=64, shuffle=False)

# I quickly inspect one sample to make sure the feature size and graph
format are as expected.
sample = train_dataset[0]
in_dim = sample.x.shape[1]
print(f"Number of training graphs: {len(train_dataset)}")
print(f"Number of test graphs: {len(test_dataset)}")
print(f"Node feature shape: {sample.x.shape}")
```

```

print(f"Edge index shape: {sample.edge_index.shape}")
print(f"Sample label: {sample.y.item()}")

Number of training graphs: 400
Number of test graphs: 100
Node feature shape: torch.Size([60, 3])
Edge index shape: torch.Size([2, 304])
Sample label: 1

# I normalize the coordinates inside each graph so the model focuses more on face shape.
# This also makes training a bit more stable for the small dataset.
def normalize_graph_features(x, batch):
    num_graphs = int(batch.max().item()) + 1
    counts = torch.bincount(batch,
minlength=num_graphs).float().unsqueeze(1).to(x.device)

    mean = torch.zeros(num_graphs, x.size(1), device=x.device)
    mean.index_add_(0, batch, x)
    mean = mean / counts.clamp_min(1.0)

    centered = x - mean[batch]
    var = torch.zeros(num_graphs, x.size(1), device=x.device)
    var.index_add_(0, batch, centered.pow(2))
    std = torch.sqrt(var / counts.clamp_min(1.0) + 1e-6)
    return centered / std[batch]

# This is a simple GraphSAGE-like layer written with plain PyTorch operations.
# For every node, it averages messages from its neighbours and combines them with the node itself.
class SimpleGraphLayer(nn.Module):
    def __init__(self, in_dim, out_dim):
        super().__init__()
        self.update = nn.Sequential(
            nn.Linear(2 * in_dim, out_dim),
            nn.ReLU(),
            nn.Linear(out_dim, out_dim),
        )

    def forward(self, x, edge_index):
        src, dst = edge_index
        messages = x[src]
        agg = torch.zeros_like(x)
        agg.index_add_(0, dst, messages)

        degree = torch.zeros(x.size(0), 1, device=x.device)
        degree.index_add_(0, dst, torch.ones(dst.size(0), 1,
device=x.device))

```

```

        mean_neighbours = agg / degree.clamp_min(1.0)

        out = self.update(torch.cat([x, mean_neighbours], dim=1))
        return F.relu(out)

# The full model first embeds node coordinates, then applies message passing.
# After that, mean and max pooling make one vector for each whole face graph.
class DrowsyGNN(nn.Module):
    def __init__(self, in_dim, hidden_dim=64, num_classes=2, dropout=0.2):
        super().__init__()
        self.encoder = nn.Sequential(
            nn.Linear(in_dim, hidden_dim),
            nn.ReLU(),
            nn.Linear(hidden_dim, hidden_dim),
        )
        self.layers = nn.ModuleList(
            [SimpleGraphLayer(hidden_dim, hidden_dim) for _ in range(3)]
        )
        self.dropout = nn.Dropout(dropout)
        self.classifier = nn.Sequential(
            nn.Linear(2 * hidden_dim, hidden_dim),
            nn.ReLU(),
            nn.Dropout(dropout),
            nn.Linear(hidden_dim, num_classes),
        )

    def forward(self, data):
        x = normalize_graph_features(data.x.float(), data.batch)
        x = F.relu(self.encoder(x))
        for layer in self.layers:
            x = layer(x, data.edge_index)
            x = self.dropout(x)
        graph_embedding = graph_mean_max_pool(x, data.batch)
        return self.classifier(graph_embedding)

# This pooling step is permutation-invariant, so it does not depend on node order.
# I use both mean and max because they capture slightly different graph information.
def graph_mean_max_pool(x, batch):
    num_graphs = int(batch.max().item()) + 1
    counts = torch.bincount(batch,
        minlength=num_graphs).float().unsqueeze(1).to(x.device)

```

```

graph_mean = torch.zeros(num_graphs, x.size(1), device=x.device)
graph_mean.index_add_(0, batch, x)
graph_mean = graph_mean / counts.clamp_min(1.0)

graph_max = []
for graph_idx in range(num_graphs):
    graph_max.append(x[batch == graph_idx].max(dim=0).values)
graph_max = torch.stack(graph_max, dim=0)

return torch.cat([graph_mean, graph_max], dim=1)

# Drowsy is the positive class here, so I compute recall and precision
# for label 1.
# I also keep TP/FP/FN/TN to show how the final metrics were
# calculated.
def compute_drowsy_metrics(y_true, y_pred):
    y_true = torch.cat(y_true).cpu()
    y_pred = torch.cat(y_pred).cpu()

    tp = ((y_pred == 1) & (y_true == 1)).sum().item()
    fn = ((y_pred == 0) & (y_true == 1)).sum().item()
    fp = ((y_pred == 1) & (y_true == 0)).sum().item()
    tn = ((y_pred == 0) & (y_true == 0)).sum().item()
    eps = 1e-8
    recall = tp / (tp + fn + eps)
    precision = tp / (tp + fp + eps)
    counts = {"tp": tp, "fp": fp, "fn": fn, "tn": tn}
    return recall, precision, counts

# One training epoch loops over the training graphs and updates the
# model weights.
# The returned loss is averaged over graphs, not over batches.
def train_one_epoch(model, loader, optimizer, criterion, device):
    model.train()
    total_loss = 0.0
    total_graphs = 0

    for batch in loader:
        batch = batch.to(device)
        y = batch.y.long().view(-1)

        optimizer.zero_grad()
        logits = model(batch)
        loss = criterion(logits, y)
        loss.backward()
        optimizer.step()

        total_loss += loss.item() * y.size(0)
        total_graphs += y.size(0)

```

```

    return total_loss / total_graphs

# Evaluation is the same forward pass, but without gradient updates.
# I collect all predictions so recall and precision are computed on
the full test set.
@torch.no_grad()
def evaluate(model, loader, criterion, device):
    model.eval()
    total_loss = 0.0
    total_graphs = 0
    all_true = []
    all_pred = []

    for batch in loader:
        batch = batch.to(device)
        y = batch.y.long().view(-1)
        logits = model(batch)
        loss = criterion(logits, y)
        pred = logits.argmax(dim=1)

        total_loss += loss.item() * y.size(0)
        total_graphs += y.size(0)
        all_true.append(y.detach().cpu())
        all_pred.append(pred.detach().cpu())

    recall, precision, counts = compute_drowsy_metrics(all_true,
all_pred)
    return total_loss / total_graphs, recall, precision, counts

# I keep the model small because there are only 400 training graphs.
# CrossEntropyLoss is used because the model outputs two logits: awake
and drowsy.
model = DrowsyGNN(in_dim=in_dim, hidden_dim=64, num_classes=2,
dropout=0.2).to(device)
criterion = nn.CrossEntropyLoss()
optimizer = torch.optim.AdamW(model.parameters(), lr=1e-3,
weight_decay=1e-4)

# I store the losses and metrics at every epoch so they can be plotted
afterwards.
history = {"train_loss": [], "test_loss": [], "recall": [],
"precision": []}
final_counts = None
epochs = 60

for epoch in range(1, epochs+1):
    train_loss = train_one_epoch(model, train_loader, optimizer,
criterion, device)

```



```

    test_loss, recall, precision, counts = evaluate(model,
test_loader, criterion, device)
    final_counts = counts

    history["train_loss"].append(train_loss)
    history["test_loss"].append(test_loss)
    history["recall"].append(recall)
    history["precision"].append(precision)

    # Printing every 10 epochs is enough to see progress without too
much output.
    if epoch == 0 or epoch % 10 == 0:
        print(
            f"epoch {epoch:03d} | train loss {train_loss:.4f} | "
            f"test loss {test_loss:.4f} | recall {recall:.4f} | "
precision {precision:.4f}"
        )

final_test_loss = history["test_loss"][-1]
final_recall = history["recall"][-1]
final_precision = history["precision"][-1]

epoch 010 | train loss 0.6027 | test loss 0.6793 | recall 0.5000 |
precision 0.6098
epoch 020 | train loss 0.5625 | test loss 0.6845 | recall 0.5200 |
precision 0.6500
epoch 030 | train loss 0.4725 | test loss 0.5776 | recall 0.7200 |
precision 0.7347
epoch 040 | train loss 0.4279 | test loss 0.5228 | recall 0.7400 |
precision 0.7872
epoch 050 | train loss 0.2982 | test loss 0.3646 | recall 0.8200 |
precision 0.8723
epoch 060 | train loss 0.2470 | test loss 0.3319 | recall 0.9000 |
precision 0.8182

# This plot compares training and test loss over the full training
run.
# A small gap near the end is expected because the model keeps fitting
the training set.
plt.figure(figsize=(7, 4))
plt.plot(history["train_loss"], label="Train loss")
plt.plot(history["test_loss"], label="Test loss")
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.title("Task 1 training and test loss")
plt.legend()
plt.grid(alpha=0.25)
plt.savefig(
    "task1_lineplot_losses.png",
    dpi=300,

```

```

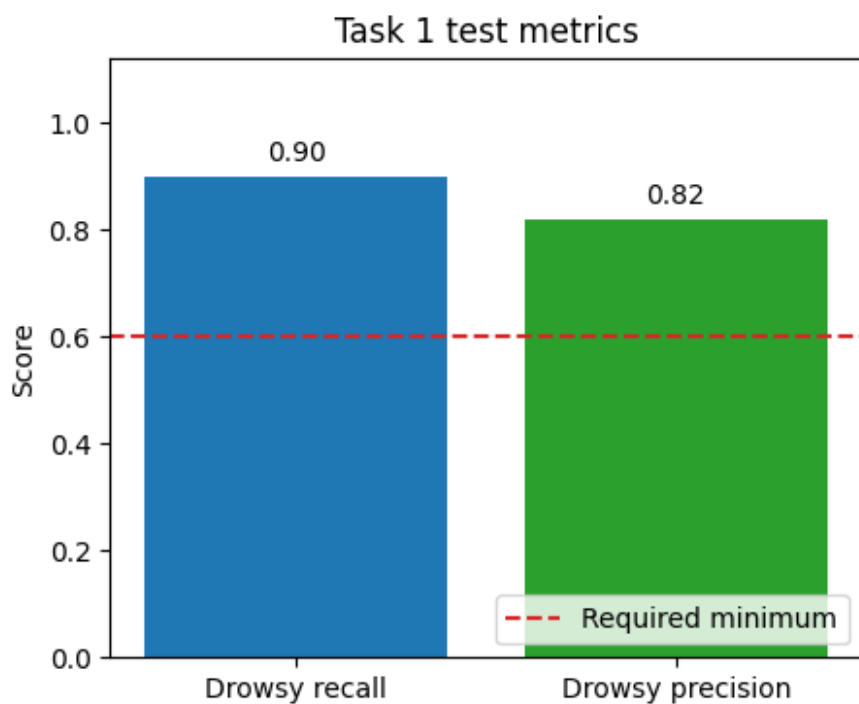
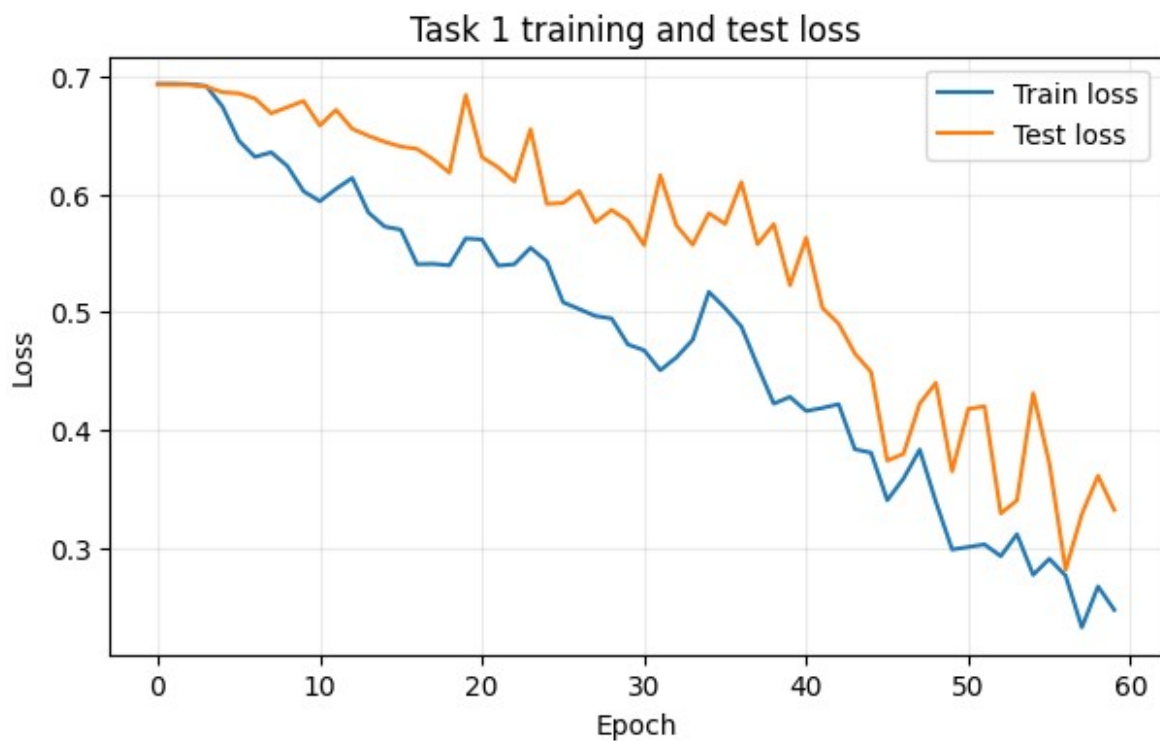
        bbox_inches="tight"
    )
plt.show()

# The metric plot shows the two required drowsy-class metrics.
# The dashed line is the assignment threshold that both bars should
pass.
plt.figure(figsize=(5, 4))
metric_names = ["Drowsy recall", "Drowsy precision"]
metric_values = [final_recall, final_precision]
plt.bar(metric_names, metric_values, color=["tab:blue", "tab:green"])
plt.axhline(0.6, linestyle="--", color="tab:red", label="Required
minimum")
plt.ylim(0, 1.12)
plt.ylabel("Score")
plt.title("Task 1 test metrics")
plt.legend(loc="lower right")
for i, value in enumerate(metric_values):
    plt.text(i, value + 0.03, f"{value:.2f}", ha="center")

plt.savefig(
    "task1_barplot_results.png",
    dpi=300,
    bbox_inches="tight"
)
plt.show()

# Final numbers are printed so they are easy to report in the
PDF/export.
print(f"Final test loss: {final_test_loss:.4f}")
print(f"Drowsy recall: {final_recall:.4f}")
print(f"Drowsy precision: {final_precision:.4f}")
print(
    "Confusion counts: "
    f"TP={final_counts['tp']}, FP={final_counts['fp']}, "
    f"FN={final_counts['fn']}, TN={final_counts['tn']}"
)

```



Final test loss: 0.3319
Drowsy recall: 0.9000
Drowsy precision: 0.8182
Confusion counts: TP=45, FP=10, FN=5, TN=40

Task 2

```
class _BaseImageClassificationDataset(Dataset):
    """
    Base dataset for image classification datasets stored as:

        dataset_dir/
            class_to_id.json
            id_to_class.json
            train/
                000000.jpg
                ...
                labels.csv
                [metadata.csv]
            test/
                000000.jpg
                ...
                labels.csv
                [metadata.csv]
    """

    def __init__(
        self,
        dataset_dir,
        split="train",
        img_transform=None,
    ):
        super().__init__()
        self.dataset_dir = Path(dataset_dir)
        self.split = split
        self.img_transform = img_transform

        if self.split not in {"train", "test"}:
            raise ValueError(f"split must be 'train' or 'test', got: {self.split}")

        self.split_dir = self.dataset_dir / self.split
        self.labels_path = self.split_dir / "labels.csv"
        self.class_to_id_path = self.dataset_dir / "class_to_id.json"
        self.id_to_class_path = self.dataset_dir / "id_to_class.json"

        if not self.dataset_dir.exists():
            raise FileNotFoundError(f"Dataset directory not found: {self.dataset_dir}")
        if not self.split_dir.exists():
            raise FileNotFoundError(f"Split directory not found: {self.split_dir}")
        if not self.labels_path.exists():
            raise FileNotFoundError(f"labels.csv not found: {self.labels_path}")
```

```

        if not self.class_to_id_path.exists():
            raise FileNotFoundError(
                f"class_to_id.json not found: {self.class_to_id_path}"
            )
        if not self.id_to_class_path.exists():
            raise FileNotFoundError(
                f"id_to_class.json not found: {self.id_to_class_path}"
            )

        with open(self.class_to_id_path, "r") as fp:
            self.class_to_id = json.load(fp)

        with open(self.id_to_class_path, "r") as fp:
            self.id_to_class = json.load(fp)

        # Normalize keys for convenience
        self.id_to_class = {int(k): v for k, v in
self.id_to_class.items()}
        self.num_classes = len(self.class_to_id)
        self.classes = [self.id_to_class[i] for i in
range(self.num_classes)]

        self.labels = (
            pd.read_csv(self.labels_path, header=None).iloc[:,
0].astype(int).tolist()
        )

        self.img_paths = sorted(
            [
                p
                for p in self.split_dir.iterdir()
                if p.is_file() and p.suffix.lower() in {".jpg",
".jpeg", ".png"}
            ],
            key=lambda p: p.name,
        )

        if len(self.img_paths) != len(self.labels):
            raise ValueError(
                f"Mismatch between number of images
({len(self.img_paths)}) and "
                f"labels ({len(self.labels)}) in split
'{self.split}'."
            )

        def __len__(self):
            return len(self.labels)

        def _load_image(self, img_path):
            with Image.open(img_path) as img:

```

```

        img = img.convert("RGB")
        if self.img_transform is not None:
            img = self.img_transform(img)
        return img

    def __getitem__(self, idx):
        img_path = self.img_paths[idx]
        image = self._load_image(img_path)
        label = self.labels[idx]

        return image, label

class BDDObjectClassificationDataset(_BaseImageClassificationDataset):
    """
    Dataset for the cropped-object classification dataset.

    Returns:
        image, label
    or:
        image, label, metadata    (if return_metadata=True)

    Notes:
        - img_transform defaults to None.
        - The user is responsible for resizing / tensor conversion.
    """

    pass

class PatchShuffle:
    def __init__(self, patch_size=8):
        self.p = patch_size

    def __call__(self, img):
        # Input: [C, H, W]
        c, h, w = img.shape
        p = self.p

        # 1. Unfold image into patches [C, H/p, W/p, p, p]
        patches = img.unfold(1, p, p).unfold(2, p, p)
        num_h = h // p
        num_w = w // p

        # 2. Reshape to [num_patches, C, p, p]
        patches = patches.contiguous().view(c, -1, p, p).permute(1, 0,
2, 3)

        # 3. Shuffle the patches along the first dimension
        idx = torch.randperm(patches.size(0))

```

```

        patches = patches[idx]

        # 4. Reshape back to original image dimensions
        # We need to rearrange from [num_h*num_w, C, p, p] back to [C,
H, W]
        patches = patches.view(num_h, num_w, c, p, p).permute(2, 0, 3,
1, 4)
        shuffled_img = patches.contiguous().view(c, h, w)

        return shuffled_img

train_dataset = BDDObjectClassificationDataset(
    dataset_dir=OBJ_CLS_DATASET_DIR,
    split="train",
    img_transform=None, # Fill train image transforms
)

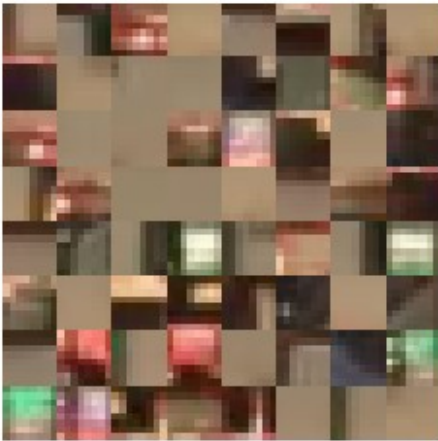
test_dataset = BDDObjectClassificationDataset(
    dataset_dir=OBJ_CLS_DATASET_DIR,
    split="test",
    img_transform=None, # Fill test image transforms
)

### See how Patch Shuffle affects an image: CODE MEANT FOR
ILLUSTRATION ###
from torchvision.transforms import Compose, ToTensor, Resize
from torchvision.transforms import functional as vf
import matplotlib.pyplot as plt

vis_dataset_noisy = BDDObjectClassificationDataset(
    dataset_dir=OBJ_CLS_DATASET_DIR,
    split="test",
    img_transform=Compose([Resize((64, 64)), ToTensor(),
PatchShuffle(patch_size=8)]),
)
vis_dataset_clean = BDDObjectClassificationDataset(
    dataset_dir=OBJ_CLS_DATASET_DIR,
    split="test",
    img_transform=Compose([Resize((64, 64)), ToTensor()]),
)
fig, ax = plt.subplots(1, 2)
img_noisy, label = vis_dataset_noisy[16]
img_clean, label = vis_dataset_clean[16]
ax[0].imshow(vf.to_pil_image(img_noisy))
ax[1].imshow(vf.to_pil_image(img_clean))
for a in ax:
    a.set_xticks([])
    a.set_yticks([])
    a.set_frame_on(False)

```

See how Patch Shuffle affects an image: CODE MEANT FOR ILLUSTRATION



```
class Lambda(nn.Module):
    def __init__(self, func, *args, **kwargs):
        super().__init__(*args, **kwargs)
        self.func = func

    def forward(self, x):
        return self.func(x)

# We used a vision transformer architecture for learning image
# representations from
# patch tokens. This architecture was chosen as the task involves
# classifying corrupted images
# with shuffled patches. Since the model processes the image as a
# sequential projection of patches,
# it is perfectly suited for the given task with minimal architectural
# changes.
class VisionTransformer(nn.Module):
    def __init__(self, n_classes, patch_size = 8, embedding_dim = 64,
num_heads = 4, mlp_dim = 128, depth = 4, dropout = 0.3):
        super().__init__()
        # This layer transforms patches into embeddings using a
        # convolution that takes a kernel_size of
        # (patch_size x patch_size). The resulting embeddings are
        # flattened because the transformer
        # expects 1 dimensional vector inputs.
        self.patch_embedding = nn.Sequential(
            nn.Conv2d(in_channels=3, out_channels=embedding_dim,
kernel_size=patch_size, stride=patch_size),
            nn.Flatten(start_dim=-2),
            Lambda(lambda x: x.transpose(-2, -1)),
        )
```



```

        # Learnable CLS token added to the patch sequence. After self-
        attention,
        # its final representation is used as the global image
        representation for classification.
        self.cls_token = nn.Parameter(torch.randn(1, 1, embedding_dim)
* 0.02)

        # Single Transformer block used to model relationships between
        patch tokens. It uses self-attention with multiple heads to create
        # multiple relations between tokens, followed by an MLP (which
        is a small neural network that processes each token representation
        independently).
        transformer_layer =
nn.TransformerEncoderLayer(d_model=embedding_dim, nhead=num_heads,
dim_feedforward=mlp_dim,

dropout=dropout, activation="gelu", norm_first=True, batch_first=True)

        self.transformer = nn.TransformerEncoder(transformer_layer,
num_layers=depth)

        # Final classification layer -> Outputs logits
        self.classification = nn.Sequential(
            nn.LayerNorm(embedding_dim),
            nn.Linear(embedding_dim, n_classes)
        )

def forward(self, x):
    patch_embedding = self.patch_embedding(x)

    batch_size = patch_embedding.size(0)

    cls_tokens = self.cls_token.expand(batch_size, -1, -1)
    patches = torch.cat([cls_tokens, patch_embedding], dim=1)

    output = self.transformer(patches)

    cls_output = output[:, 0]

    return self.classification(cls_output)

train_task2 = True # Set to false for loading saved model
infer_task2 = True

# Model trainer that uses a checkpointing strategy to keep the best
performing
# model during training.
class VisionTransformerTrainer():
    def __init__(
        self,

```

```

model: torch.nn.Module,
device: torch.device,
criterion: torch.nn.Module,
optimizer: torch.optim.Optimizer,
training_DataLoader: torch.utils.data.DataLoader,
validation_DataLoader: torch.utils.data.DataLoader,
epochs: int,
criterion_validation: torch.nn.Module = None,
checkpoint: bool = False,
checkpoint_path: str = "best_vit_model.pt",
):
    self.model = model
    self.criterion = criterion
    self.optimizer = optimizer
    self.training_DataLoader = training_DataLoader
    self.validation_DataLoader = validation_DataLoader
    self.device = device
    self.epochs = epochs
    self.checkpoint = checkpoint
    if self.checkpoint:
        self.checkpoint_path = checkpoint_path

    if criterion_validation is None:
        self.criterion_validation = criterion
    else:
        self.criterion_validation = criterion_validation

def run_trainer(self):
    best_val_acc = 0.0
    best_val_loss = float("inf")
    best_epoch = 0
    best_model_state = None

    for epoch in tqdm(range(self.epochs)):
        self.model.train()

        train_losses = []
        correct = 0
        total = 0

        for batch in self.training_DataLoader:
            x, label_x = batch

            input = x.to(self.device)
            label_target = label_x.to(self.device).long()

            self.optimizer.zero_grad()

            out = self.model(input)
            loss = self.criterion(out, label_target)

```

```

        loss.backward()
        self.optimizer.step()

        train_losses.append(loss.item())

        preds = out.argmax(dim=1)
        correct += (preds == label_target).sum().item()
        total += label_target.size(0)

    train_loss = np.mean(train_losses)
    train_acc = correct / total

    self.model.eval()

    valid_losses = []
    valid_correct = 0
    valid_total = 0

    with torch.no_grad():
        for batch in self.validation_DataLoader:
            x, label_x = batch

            input = x.to(self.device)
            label_target = label_x.to(self.device).long()

            out = self.model(input)
            loss = self.criterion_validation(out,
label_target)

            valid_losses.append(loss.item())

            preds = out.argmax(dim=1)
            valid_correct += (preds ==
label_target).sum().item()
            valid_total += label_target.size(0)

    val_loss = np.mean(valid_losses)
    val_acc = valid_correct / max(valid_total, 1)

    if self.checkpoint and val_acc > best_val_acc:
        best_val_acc = val_acc
        best_val_loss = val_loss
        best_epoch = epoch + 1

        best_model_state =
copy.deepcopy(self.model.state_dict())

        torch.save({
            "epoch": best_epoch,
            "model_state_dict": self.model.state_dict(),

```

```

        "optimizer_state_dict":
self.optimizer.state_dict(),
        "val_acc": best_val_acc,
        "val_loss": best_val_loss,
    }, self.checkpoint_path)

    print(
        f"EPOCH: {epoch +
1:0>{len(str(self.epochs))}}/{self.epochs} "
        f"LOSS: {train_loss:.4f} "
        f"VAL-LOSS: {val_loss:.4f}"
    )
    print(f"ACCURACY: {train_acc:.4f}")
    print(f"VAL-ACCURACY: {val_acc:.4f}")

    print(f"Training finished.")

    if self.checkpoint and best_model_state is not None :
        self.model.load_state_dict(best_model_state)
        print(f"Best model loaded from epoch: {best_epoch} with
val accuracy: {best_val_acc}.")

    def load_last_model(self):
        checkpoint = torch.load(self.checkpoint_path,
map_location=DEVICE, weights_only=False)
        self.model.load_state_dict(checkpoint["model_state_dict"])
        print("Last model loaded.")

# Dataset preparation
# SAME FOR TASK 2 AND 3
transform = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229,
0.224, 0.225])
])

# Datasets with transformations (uniform sizing)
train_dataset = BDDObjectClassificationDataset(
    dataset_dir=OBJ_CLS_DATASET_DIR,
    split="train",
    img_transform=transform,
)

test_dataset = BDDObjectClassificationDataset(
    dataset_dir=OBJ_CLS_DATASET_DIR,
    split="test",
    img_transform=transform,
)

```

```

validate_dataset = BDDObjectClassificationDataset(
    dataset_dir=OBJ_CLS_DATASET_DIR,
    split="test",
    img_transform=transform,
)

def evaluate_per_class(model, loader, class_names, labels, visualize =
False):
    model.eval()

    all_preds = []
    all_labels = []

    with torch.inference_mode():
        for x, y in loader:
            x = x.to(DEVICE)
            y = y.to(DEVICE).long()

            out = model(x)
            preds = out.argmax(dim=1)

            all_preds.extend(preds.cpu().numpy())
            all_labels.extend(y.cpu().numpy())

    all_preds = np.array(all_preds)
    all_labels = np.array(all_labels)

    per_class_precision = []
    per_class_recall = []

    for label in set(labels):
        tp = np.sum((all_preds == label) & (all_labels == label))
        fp = np.sum((all_preds == label) & (all_labels != label))
        fn = np.sum((all_preds != label) & (all_labels == label))

        if tp + fp == 0:
            precision = 0.0
        else:
            precision = tp / (tp + fp)

        if tp + fn == 0:
            recall = 0.0
        else:
            recall = tp / (tp + fn)

        per_class_precision.append(precision)
        per_class_recall.append(recall)

    if visualize:
        print(f"{class_names[label]:15s} | Precision:

```

```

{precision:.4f} | Recall: {recall:.4f}")

    # Visualization in a single figure
    if visualize:

        x = np.arange(n_classes)
        width = 0.35

        plt.figure(figsize=(12, 6))

        plt.bar(x - width / 2, per_class_precision, width,
label="Precision")
        plt.bar(x + width / 2, per_class_recall, width,
label="Recall")

        plt.xticks(x, class_names)
        plt.ylim(0, 1)
        plt.ylabel("Score")
        plt.title("Per-class Precision and Recall")
        plt.legend()
        plt.tight_layout()
        plt.savefig("per_class_precision_recall.png", dpi=300,
bbox_inches="tight")
        plt.show()

    return per_class_precision, per_class_recall

def aggregate_per_class_metrics(per_class_precision, per_class_recall):
    precision_macro = np.mean(per_class_precision)
    recall_macro = np.mean(per_class_recall)

    print()
    print(f"Macro precision: {precision_macro:.4f}")
    print(f"Macro recall:      {recall_macro:.4f}")

    return precision_macro, recall_macro

train_data_loader = DataLoader(train_dataset, batch_size=32,
shuffle=True)
test_data_loader = DataLoader(test_dataset, batch_size=32,
shuffle=False)

patch_size = train_dataset[0][0].size(-1) // 8
n_classes = len(set(train_dataset.labels))

# Based on experimental observations, I arrived at the conclusion that
a smaller model performs better for the given task.
# Larger models tend to overfit, most likely due to the small size of
the dataset. Another measure used to reduce overfitting
# is increasing weight decay and dropout. Weight decay penalizes large

```

model weights and dropout randomly disables part of the activations during training. Further experiments could be performed to arrive at a better model, but as of its current configuration, this model achieves a training accuracy of ~0.60 and test accuracy between 0.45 and 0.5.

```
model = VisionTransformer(n_classes=n_classes, patch_size=patch_size,
embedding_dim=64, num_heads=4, mlp_dim=256, depth=2,
dropout=0.25).to(DEVICE)
```

This loss function is used as the model outputs logits. It is also the standard criterion for single-label multi-class classification.

```
criterion = nn.CrossEntropyLoss()
```

AdamW optimizer with a small learning rate for stable training.

Weight decay provides regularization to reduce overfitting.

```
optimizer = torch.optim.AdamW(model.parameters(), lr=1e-4,
weight_decay=2e-3)
```

```
trainer = VisionTransformerTrainer(
    model,
    DEVICE,
    criterion,
    optimizer,
    train_data_loader,
    test_data_loader,
    epochs=100,
    checkpoint_path="best_vit_model_v2.pt",
    checkpoint=True
)
```

C:\Users\20223084\AppData\Local\Temp\ipykernel_19980\3351505348.py:34:

UserWarning: enable_nested_tensor is True, but self.use_nested_tensor is False because encoder_layer.norm_first was True

```
self.transformer = nn.TransformerEncoder(transformer_layer,
num_layers=depth)
```

```
if train_task2:
    trainer.run_trainer()
else:
    trainer.load_last_model()
```

```
1%|█
| 1/100 [00:06<09:57, 6.03s/it]
```

```
EPOCH: 001/100 LOSS: 2.0779 VAL-LOSS: 1.9713
ACCURACY: 0.1675
VAL-ACCURACY: 0.2125
```

```
2%|█
| 2/100 [00:10<08:24, 5.15s/it]
```

EPOCH: 002/100 LOSS: 2.0223 VAL-LOSS: 1.9584
ACCURACY: 0.1900
VAL-ACCURACY: 0.2417

3% | 
| 3/100 [00:15<07:54, 4.89s/it]

EPOCH: 003/100 LOSS: 1.9871 VAL-LOSS: 1.9470
ACCURACY: 0.2146
VAL-ACCURACY: 0.2625

4% | 
| 4/100 [00:19<07:44, 4.83s/it]

EPOCH: 004/100 LOSS: 1.9726 VAL-LOSS: 1.9405
ACCURACY: 0.2100
VAL-ACCURACY: 0.2583

5% | 
| 5/100 [00:24<07:18, 4.62s/it]

EPOCH: 005/100 LOSS: 1.9429 VAL-LOSS: 1.8754
ACCURACY: 0.2446
VAL-ACCURACY: 0.2542

6% | 
| 6/100 [00:28<07:12, 4.60s/it]

EPOCH: 006/100 LOSS: 1.9180 VAL-LOSS: 1.8670
ACCURACY: 0.2387
VAL-ACCURACY: 0.2833

7% | 
| 7/100 [00:33<07:03, 4.56s/it]

EPOCH: 007/100 LOSS: 1.8902 VAL-LOSS: 1.8348
ACCURACY: 0.2575
VAL-ACCURACY: 0.2958

8% | 
| 8/100 [00:38<07:12, 4.70s/it]

EPOCH: 008/100 LOSS: 1.8600 VAL-LOSS: 1.8271
ACCURACY: 0.2621
VAL-ACCURACY: 0.2583

9% | 
| 9/100 [00:43<07:12, 4.76s/it]

EPOCH: 009/100 LOSS: 1.8598 VAL-LOSS: 1.8733
ACCURACY: 0.2654
VAL-ACCURACY: 0.2750

10% | ██████████
| 10/100 [00:47<07:03, 4.71s/it]

EPOCH: 010/100 LOSS: 1.8296 VAL-LOSS: 1.7922
ACCURACY: 0.2925
VAL-ACCURACY: 0.3083

11% | ██████████
| 11/100 [00:52<06:56, 4.68s/it]

EPOCH: 011/100 LOSS: 1.8066 VAL-LOSS: 1.7753
ACCURACY: 0.2913
VAL-ACCURACY: 0.3000

12% | ██████████
| 12/100 [00:56<06:52, 4.69s/it]

EPOCH: 012/100 LOSS: 1.8000 VAL-LOSS: 1.8278
ACCURACY: 0.2950
VAL-ACCURACY: 0.2875

13% | ██████████
| 13/100 [01:01<06:43, 4.64s/it]

EPOCH: 013/100 LOSS: 1.7843 VAL-LOSS: 1.7970
ACCURACY: 0.3113
VAL-ACCURACY: 0.3042

14% | ██████████
| 14/100 [01:06<06:37, 4.62s/it]

EPOCH: 014/100 LOSS: 1.7573 VAL-LOSS: 1.8083
ACCURACY: 0.3300
VAL-ACCURACY: 0.2875

15% | ██████████
| 15/100 [01:10<06:32, 4.62s/it]

EPOCH: 015/100 LOSS: 1.7419 VAL-LOSS: 1.7917
ACCURACY: 0.3317
VAL-ACCURACY: 0.2792

16% | ██████████
| 16/100 [01:15<06:33, 4.68s/it]

EPOCH: 016/100 LOSS: 1.7226 VAL-LOSS: 1.7366
ACCURACY: 0.3396
VAL-ACCURACY: 0.3167

17% | ██████████
| 17/100 [01:20<06:26, 4.65s/it]

EPOCH: 017/100 LOSS: 1.7150 VAL-LOSS: 1.6984
ACCURACY: 0.3404
VAL-ACCURACY: 0.3292

18% | ██████████
| 18/100 [01:24<06:25, 4.70s/it]

EPOCH: 018/100 LOSS: 1.6803 VAL-LOSS: 1.6931
ACCURACY: 0.3533
VAL-ACCURACY: 0.3583

19% | ██████████
| 19/100 [01:29<06:25, 4.76s/it]

EPOCH: 019/100 LOSS: 1.6735 VAL-LOSS: 1.7344
ACCURACY: 0.3658
VAL-ACCURACY: 0.3333

20% | ██████████
| 20/100 [01:34<06:26, 4.83s/it]

EPOCH: 020/100 LOSS: 1.6701 VAL-LOSS: 1.6989
ACCURACY: 0.3733
VAL-ACCURACY: 0.3917

21% | ██████████
| 21/100 [01:40<06:31, 4.96s/it]

EPOCH: 021/100 LOSS: 1.6503 VAL-LOSS: 1.7196
ACCURACY: 0.3725
VAL-ACCURACY: 0.3458

22% | ██████████
| 22/100 [01:44<06:17, 4.84s/it]

EPOCH: 022/100 LOSS: 1.6418 VAL-LOSS: 1.6851
ACCURACY: 0.3692
VAL-ACCURACY: 0.3625

23% | ██████████
| 23/100 [01:49<06:08, 4.78s/it]

EPOCH: 023/100 LOSS: 1.6273 VAL-LOSS: 1.6553
ACCURACY: 0.3821
VAL-ACCURACY: 0.3667

24% | ██████████
| 24/100 [01:53<05:58, 4.71s/it]

EPOCH: 024/100 LOSS: 1.6296 VAL-LOSS: 1.6520
ACCURACY: 0.3762
VAL-ACCURACY: 0.3708

25%|██████████
| 25/100 [01:58<05:54, 4.72s/it]

EPOCH: 025/100 LOSS: 1.6079 VAL-LOSS: 1.7089
ACCURACY: 0.3821
VAL-ACCURACY: 0.3542

26%|██████████
| 26/100 [02:04<06:04, 4.93s/it]

EPOCH: 026/100 LOSS: 1.6101 VAL-LOSS: 1.6406
ACCURACY: 0.3879
VAL-ACCURACY: 0.3958

27%|██████████
| 27/100 [02:09<06:18, 5.19s/it]

EPOCH: 027/100 LOSS: 1.5956 VAL-LOSS: 1.6844
ACCURACY: 0.3946
VAL-ACCURACY: 0.3833

28%|██████████
| 28/100 [02:15<06:15, 5.22s/it]

EPOCH: 028/100 LOSS: 1.5972 VAL-LOSS: 1.6682
ACCURACY: 0.3887
VAL-ACCURACY: 0.3708

29%|██████████
| 29/100 [02:19<05:58, 5.05s/it]

EPOCH: 029/100 LOSS: 1.5793 VAL-LOSS: 1.6619
ACCURACY: 0.4050
VAL-ACCURACY: 0.3917

30%|██████████
| 30/100 [02:24<05:42, 4.90s/it]

EPOCH: 030/100 LOSS: 1.5593 VAL-LOSS: 1.6605
ACCURACY: 0.4129
VAL-ACCURACY: 0.3625

31%|██████████
| 31/100 [02:29<05:51, 5.10s/it]

EPOCH: 031/100 LOSS: 1.5604 VAL-LOSS: 1.6687
ACCURACY: 0.4175
VAL-ACCURACY: 0.4000

32%|██████████
| 32/100 [02:37<06:35, 5.81s/it]

EPOCH: 032/100 LOSS: 1.5435 VAL-LOSS: 1.6553
ACCURACY: 0.4250
VAL-ACCURACY: 0.4125

EPOCH: 033/100 LOSS: 1.5415 VAL-LOSS: 1.6293
ACCURACY: 0.4279
VAL-ACCURACY: 0.3792

EPOCH: 034/100 LOSS: 1.5507 VAL-LOSS: 1.6439
ACCURACY: 0.4075
VAL-ACCURACY: 0.4208

EPOCH: 035/100 LOSS: 1.5243 VAL-LOSS: 1.6649
ACCURACY: 0.4221
VAL-ACCURACY: 0.4000

EPOCH: 036/100 LOSS: 1.5146 VAL-LOSS: 1.6296
ACCURACY: 0.4317
VAL-ACCURACY: 0.4375

EPOCH: 037/100 LOSS: 1.5014 VAL-LOSS: 1.6499
ACCURACY: 0.4396
VAL-ACCURACY: 0.4042

EPOCH: 038/100 LOSS: 1.4999 VAL-LOSS: 1.6330
ACCURACY: 0.4417
VAL-ACCURACY: 0.4292

EPOCH: 039/100 LOSS: 1.5111 VAL-LOSS: 1.6399
ACCURACY: 0.4363
VAL-ACCURACY: 0.4292

EPOCH: 062/100 LOSS: 1.3897 VAL-LOSS: 1.6434
ACCURACY: 0.4775
VAL-ACCURACY: 0.4417

63% | 
| 63/100 [05:28<03:11, 5.18s/it]

EPOCH: 063/100 LOSS: 1.3631 VAL-LOSS: 1.6706
ACCURACY: 0.4888
VAL-ACCURACY: 0.4333

64% | 
| 64/100 [05:35<03:20, 5.57s/it]

EPOCH: 064/100 LOSS: 1.3583 VAL-LOSS: 1.6411
ACCURACY: 0.5008
VAL-ACCURACY: 0.4417

65% | 
| 65/100 [05:40<03:12, 5.50s/it]

EPOCH: 065/100 LOSS: 1.3714 VAL-LOSS: 1.6546
ACCURACY: 0.4875
VAL-ACCURACY: 0.3917

66% | 
| 66/100 [05:45<03:03, 5.39s/it]

EPOCH: 066/100 LOSS: 1.3317 VAL-LOSS: 1.6292
ACCURACY: 0.4963
VAL-ACCURACY: 0.4625

67% | 
| 67/100 [05:50<02:57, 5.39s/it]

EPOCH: 067/100 LOSS: 1.3578 VAL-LOSS: 1.6234
ACCURACY: 0.5017
VAL-ACCURACY: 0.4542

68% | 
| 68/100 [05:55<02:48, 5.26s/it]

EPOCH: 068/100 LOSS: 1.3522 VAL-LOSS: 1.6391
ACCURACY: 0.5154
VAL-ACCURACY: 0.4042

69% | 
| 69/100 [06:01<02:42, 5.25s/it]

EPOCH: 069/100 LOSS: 1.3455 VAL-LOSS: 1.6512
ACCURACY: 0.5025
VAL-ACCURACY: 0.4333

70%|

| 70/100 [06:08<02:59, 5.97s/it]

EPOCH: 070/100 LOSS: 1.3433 VAL-LOSS: 1.6498

ACCURACY: 0.5054

VAL-ACCURACY: 0.4167

71%|

| 71/100 [06:14<02:47, 5.79s/it]

EPOCH: 071/100 LOSS: 1.3343 VAL-LOSS: 1.6193

ACCURACY: 0.5008

VAL-ACCURACY: 0.4458

72%|

| 72/100 [06:18<02:34, 5.53s/it]

EPOCH: 072/100 LOSS: 1.3405 VAL-LOSS: 1.6369

ACCURACY: 0.4938

VAL-ACCURACY: 0.4042

73%|

| 73/100 [06:25<02:37, 5.84s/it]

EPOCH: 073/100 LOSS: 1.3382 VAL-LOSS: 1.7150

ACCURACY: 0.4983

VAL-ACCURACY: 0.4167

74%|

| 74/100 [06:31<02:32, 5.87s/it]

EPOCH: 074/100 LOSS: 1.3296 VAL-LOSS: 1.6297

ACCURACY: 0.5067

VAL-ACCURACY: 0.4333

75%|

| 75/100 [06:36<02:22, 5.72s/it]

EPOCH: 075/100 LOSS: 1.3154 VAL-LOSS: 1.6416

ACCURACY: 0.5092

VAL-ACCURACY: 0.4208

76%|

| 76/100 [06:41<02:12, 5.54s/it]

EPOCH: 076/100 LOSS: 1.3141 VAL-LOSS: 1.6497

ACCURACY: 0.5158

VAL-ACCURACY: 0.4583

77%|

| 77/100 [06:47<02:05, 5.46s/it]

EPOCH: 077/100 LOSS: 1.3075 VAL-LOSS: 1.6347
ACCURACY: 0.5067
VAL-ACCURACY: 0.4542

78% | 
| 78/100 [06:52<01:56, 5.29s/it]

EPOCH: 078/100 LOSS: 1.3028 VAL-LOSS: 1.6697
ACCURACY: 0.5233
VAL-ACCURACY: 0.4208

79% | 
| 79/100 [06:57<01:49, 5.23s/it]

EPOCH: 079/100 LOSS: 1.3302 VAL-LOSS: 1.6101
ACCURACY: 0.5004
VAL-ACCURACY: 0.4375

80% | 
| 80/100 [07:02<01:43, 5.17s/it]

EPOCH: 080/100 LOSS: 1.2987 VAL-LOSS: 1.6341
ACCURACY: 0.5204
VAL-ACCURACY: 0.4333

81% | 
| 81/100 [07:07<01:41, 5.34s/it]

EPOCH: 081/100 LOSS: 1.2976 VAL-LOSS: 1.6531
ACCURACY: 0.5221
VAL-ACCURACY: 0.4292

82% | 
| 82/100 [07:12<01:34, 5.23s/it]

EPOCH: 082/100 LOSS: 1.2766 VAL-LOSS: 1.6901
ACCURACY: 0.5288
VAL-ACCURACY: 0.4083

83% | 
| 83/100 [07:19<01:33, 5.50s/it]

EPOCH: 083/100 LOSS: 1.2804 VAL-LOSS: 1.6233
ACCURACY: 0.5238
VAL-ACCURACY: 0.4333

84% | 
| 84/100 [07:24<01:29, 5.62s/it]

EPOCH: 084/100 LOSS: 1.2802 VAL-LOSS: 1.6383
ACCURACY: 0.5233
VAL-ACCURACY: 0.4417

85%|

| 85/100 [07:30<01:23, 5.59s/it]

EPOCH: 085/100 LOSS: 1.2815 VAL-LOSS: 1.6277
ACCURACY: 0.5192
VAL-ACCURACY: 0.4292

86%|

| 86/100 [07:35<01:17, 5.54s/it]

EPOCH: 086/100 LOSS: 1.2908 VAL-LOSS: 1.6741
ACCURACY: 0.5229
VAL-ACCURACY: 0.4042

87%|

| 87/100 [07:42<01:14, 5.70s/it]

EPOCH: 087/100 LOSS: 1.2640 VAL-LOSS: 1.6306
ACCURACY: 0.5242
VAL-ACCURACY: 0.4667

88%|

| 88/100 [07:47<01:08, 5.71s/it]

EPOCH: 088/100 LOSS: 1.2550 VAL-LOSS: 1.6319
ACCURACY: 0.5450
VAL-ACCURACY: 0.4333

89%|

| 89/100 [07:52<01:00, 5.48s/it]

EPOCH: 089/100 LOSS: 1.2436 VAL-LOSS: 1.6667
ACCURACY: 0.5375
VAL-ACCURACY: 0.4167

90%|

| 90/100 [07:59<00:58, 5.86s/it]

EPOCH: 090/100 LOSS: 1.2409 VAL-LOSS: 1.6756
ACCURACY: 0.5437
VAL-ACCURACY: 0.4167

91%|

| 91/100 [08:06<00:56, 6.24s/it]

EPOCH: 091/100 LOSS: 1.2508 VAL-LOSS: 1.6363

ACCURACY: 0.5354

VAL-ACCURACY: 0.4417

92%|

| 92/100 [08:11<00:47, 5.96s/it]

EPOCH: 092/100 LOSS: 1.2535 VAL-LOSS: 1.6524

ACCURACY: 0.5354

VAL-ACCURACY: 0.4375

93%|

| 93/100 [08:17<00:40, 5.85s/it]

EPOCH: 093/100 LOSS: 1.2324 VAL-LOSS: 1.6503

ACCURACY: 0.5500

VAL-ACCURACY: 0.4333

94%|

| 94/100 [08:22<00:34, 5.74s/it]

EPOCH: 094/100 LOSS: 1.2298 VAL-LOSS: 1.6675

ACCURACY: 0.5475

VAL-ACCURACY: 0.4625

95%|

| 95/100 [08:27<00:27, 5.47s/it]

EPOCH: 095/100 LOSS: 1.2481 VAL-LOSS: 1.6791

ACCURACY: 0.5304

VAL-ACCURACY: 0.4208

96%|

| 96/100 [08:33<00:22, 5.67s/it]

EPOCH: 096/100 LOSS: 1.2482 VAL-LOSS: 1.6557

ACCURACY: 0.5417

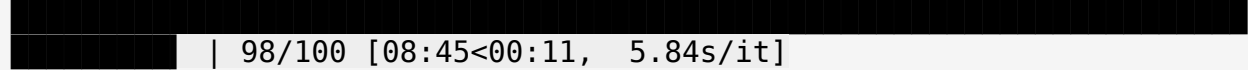
VAL-ACCURACY: 0.4208

97%|

| 97/100 [08:38<00:16, 5.48s/it]

EPOCH: 097/100 LOSS: 1.2303 VAL-LOSS: 1.6797
ACCURACY: 0.5442
VAL-ACCURACY: 0.4333

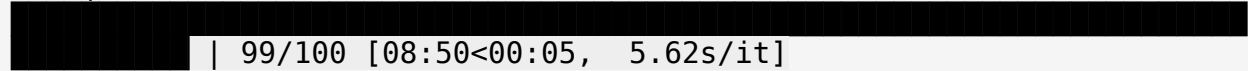
98%|



| 98/100 [08:45<00:11, 5.84s/it]

EPOCH: 098/100 LOSS: 1.2230 VAL-LOSS: 1.6498
ACCURACY: 0.5379
VAL-ACCURACY: 0.4417

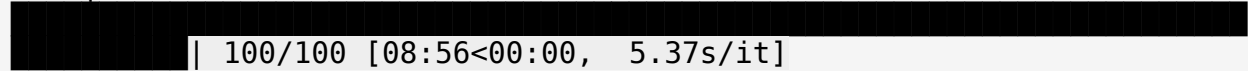
99%|



| 99/100 [08:50<00:05, 5.62s/it]

EPOCH: 099/100 LOSS: 1.2317 VAL-LOSS: 1.6774
ACCURACY: 0.5517
VAL-ACCURACY: 0.4208

100%|



| 100/100 [08:56<00:00, 5.37s/it]

EPOCH: 100/100 LOSS: 1.2192 VAL-LOSS: 1.6305
ACCURACY: 0.5625
VAL-ACCURACY: 0.4542
Training finished.
Best model loaded from epoch: 87 with val accuracy:
0.4666666666666667.

Per class evaluation on the corrupted dataset (shuffled patches)

validation_data_loader = DataLoader(dataset=validate_dataset,
batch_size=32)

per_class_precision_corrupted, per_class_recall_corrupted =
evaluate_per_class(model, validation_data_loader,

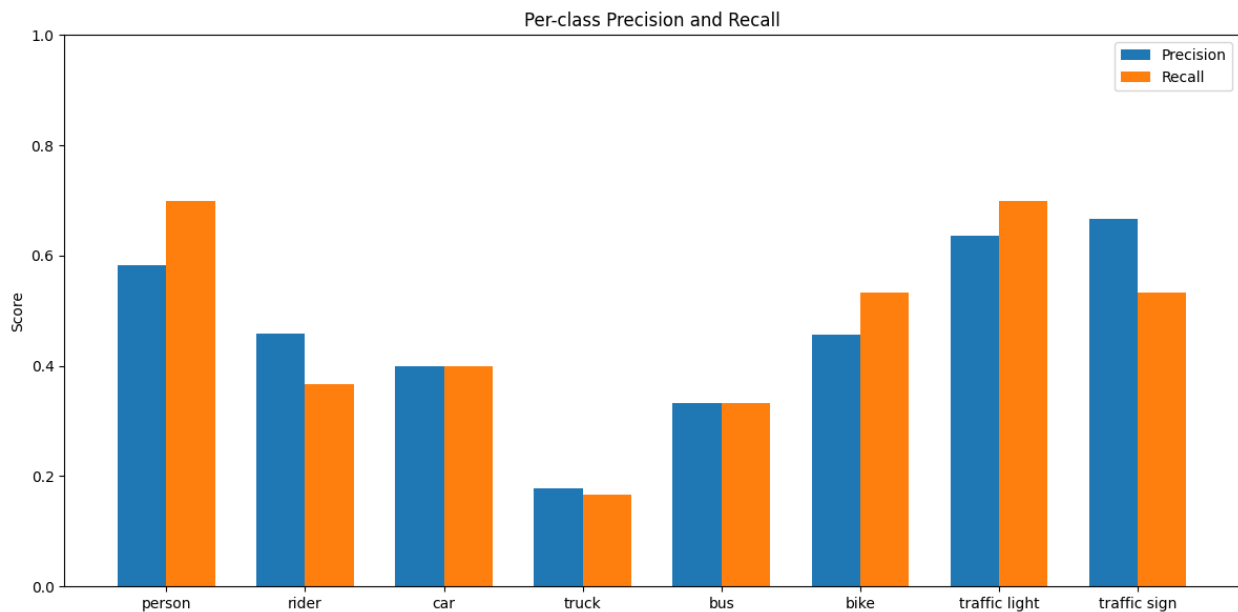
validate_dataset.classes, validate_dataset.labels, visualize=True)

*# Model performs best for the "traffic_light" class with a precision
and recall of 0.66*

*# Model performs worst for the "bus" class (precision=0.26,
recall=0.26) and the "truck" class (precision=0.36, recall=0.23)*

person	Precision: 0.5833	Recall: 0.7000
rider	Precision: 0.4583	Recall: 0.3667
car	Precision: 0.4000	Recall: 0.4000
truck	Precision: 0.1786	Recall: 0.1667
bus	Precision: 0.3333	Recall: 0.3333

bike	Precision: 0.4571	Recall: 0.5333
traffic light	Precision: 0.6364	Recall: 0.7000
traffic sign	Precision: 0.6667	Recall: 0.5333



```
# Here, the agregated metrics are the same for both the corrupted
dataset and the clean dataset.
# This means that the model is invariant to the positions of the
patches, and as a result, predicts the
# same labels.
print("Agregated metrics for corrupted dataset:")
# Macro Prec = The mean per-class precision scores | Macro Recall =
Same but for recall
macro_prec_corrupt, macro_recall_corrupt =
aggregate_per_class_metrics(per_class_precision_corrupted,
per_class_recall_corrupted)

per_class_precision_clean, per_class_recall_clean =
evaluate_per_class(model, test_data_loader, test_dataset.classes,
test_dataset.labels)

print("\nAgregated metrics for clean dataset:")
macro_prec_clean, macro_recall_clean =
aggregate_per_class_metrics(per_class_precision_clean,
per_class_recall_clean)
```

Agregated metrics for corrupted dataset:

Macro precision: 0.4642
Macro recall: 0.4667

Agregated metrics for clean dataset:

Macro precision: 0.4642

Macro recall: 0.4667

Visualization

x = np.arange(2)

width = 0.35

precision_values = np.array([macro_prec_corrupt, macro_prec_clean])

recall_values = np.array([macro_recall_corrupt, macro_recall_clean])

plt.figure(figsize=(12, 6))

plt.bar(x - width / 2, precision_values, width, label="Precision")

plt.bar(x + width / 2, recall_values, width, label="Recall")

Baseline of 0.3 as described by the task

```
plt.axhline(
    y=0.3,
    linestyle="dotted",
    linewidth=2,
    color="red",
    label=f"Baseline ({0.3:.2f})"
)
```

plt.xticks(x, ["Corrupt Dataset", "Clean Dataset"])

plt.ylim(0, 1)

plt.ylabel("Score")

plt.title("Macro Precision and Recall")

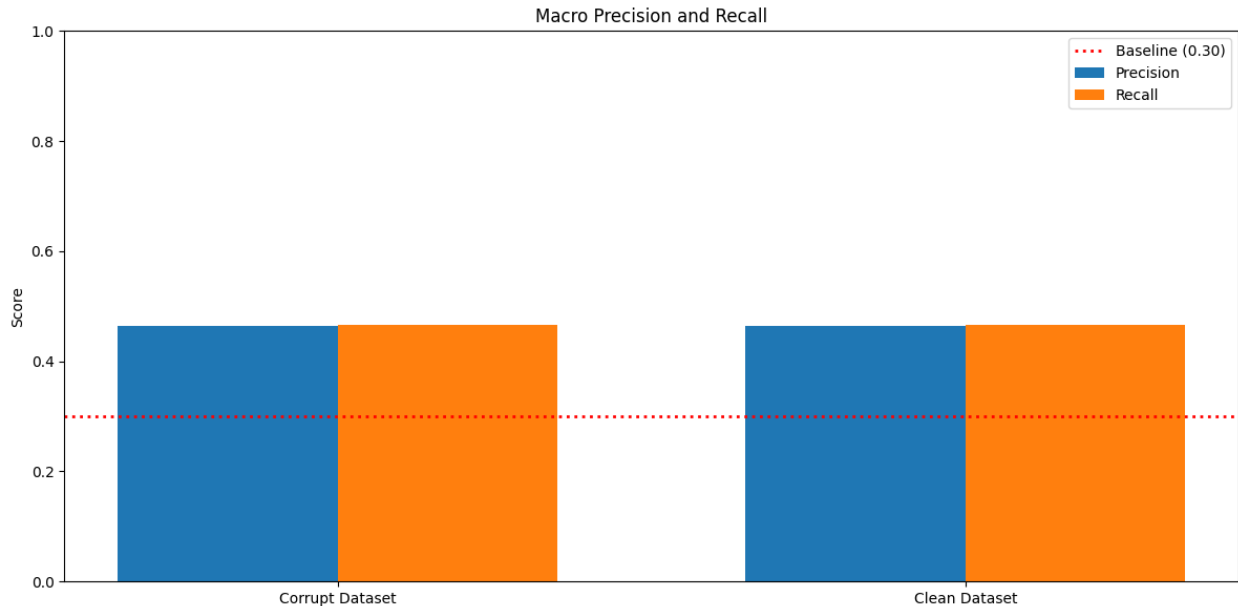
plt.legend()

plt.tight_layout()

plt.savefig("macro_precision_recall_comparison.png", dpi=300,

bbox_inches="tight")

plt.show()



Task 3

```
# Get indices of where classes begin/end
class_to_idx = defaultdict(list)

for idx in range(len(train_dataset)):
    _, label = train_dataset[idx]
    class_name = train_dataset.id_to_class[label]
    class_to_idx[class_name].append(idx)

# Denormalize before plotting
def denormalize(img):
    mean = torch.tensor([0.485, 0.456, 0.406]).view(3,1,1)
    std = torch.tensor([0.229, 0.224, 0.225]).view(3,1,1)
    return img * std + mean

# Plot one sample of each class
fig, axes = plt.subplots(2, 4, figsize=(12, 6))
axes = axes.flatten()

classes = sorted(train_dataset.classes)

for i, cls in enumerate(classes):
    idx = class_to_idx[cls][16]

    img_pre, label = train_dataset[idx]
    img = denormalize(img_pre).clamp(0, 1)

    img = img.permute(1, 2, 0).cpu().numpy()

    axes[i].imshow(img)
    axes[i].set_title(cls)
```



```
axes[i].axis('off')
plt.tight_layout()
plt.show()
```



```
# This utility class is aimed to help you a bit with the linear probe classifier.
# You can choose whether to implement it or not!
```

```
class LinearProbeClassifier(nn.Module):
    """
    Frozen pretrained backbone + trainable linear classification head.

    TODO:
    - load a pretrained backbone,
    - remove/replace its original classification head (if any),
    - freeze the backbone parameters,
    - add a new trainable linear head for the BDD classes.
    """

    def __init__(self, num_classes: int = 8):
        super().__init__()
        # Load pretrained model backbone (ignore backbone_arch string)
        self.backbone = SwinModel.from_pretrained("microsoft/swin-
large-patch4-window7-224")

        # TODO: Sometimes the pretrained backbone may already have a
        classification head (e.g. ResNet's fc layer, ViT's heads layer).
        # In this case you might need to replace it with an Identity
```

```

layer to get the features before the head.
    # The Identity layer is simply a placeholder for the Identity
function, which returns its input unchanged.
    # --> SwinModel loads without a classification head

    # Get hidden dimension size for linear head
    hidden_size = self.backbone.config.hidden_size

    # TODO: freeze all pretrained backbone parameters. Freezing a
parameter means that it will not be updated during gradient updates.
    for param in self.backbone.parameters():
        param.requires_grad = False

    # TODO: define the trainable linear classification head
    self.linear_head = nn.Linear(hidden_size, num_classes)

def forward(self, x: torch.Tensor):
    # The backbone should be used as a feature extractor.
    # No gradient tracking during forward
    with torch.no_grad():
        outputs = self.backbone(x)

    # Avg pooling on last layer hidden-state
    features = outputs.pooler_output

    # Then the features from the backbone are passed through the
linear head to get the final logits for classification.
    logits = self.linear_head(features)

    return logits

def compute_macro_metrics(y_true, y_pred, num_classes=8):
    """Compute macro precision and recall over num_classes classes."""
    y_true = torch.cat(y_true).cpu()
    y_pred = torch.cat(y_pred).cpu()

    eps = 1e-8 # epsilon value to avoid div by zero
    precision_per_class = []
    recall_per_class = []

    # Get per-class precision and recall
    for c in range(num_classes):
        tp = ((y_pred == c) & (y_true == c)).sum().item()
        fp = ((y_pred == c) & (y_true != c)).sum().item()
        fn = ((y_pred != c) & (y_true == c)).sum().item()

        precision = tp / (tp + fp + eps)
        recall = tp / (tp + fn + eps)

        precision_per_class.append(precision)

```

```

        recall_per_class.append(recall)

    macro_precision = np.mean(precision_per_class)
    macro_recall = np.mean(recall_per_class)

    return macro_precision, macro_recall

# This code is taken and adapted from Practical P2.2.
class Trainer_task3():
    def __init__(self,
                  model: torch.nn.Module,
                  device: torch.device,
                  criterion: torch.nn.Module,
                  optimizer: torch.optim.Optimizer,
                  training_DataLoader: torch.utils.data.Dataset,
                  validation_DataLoader: torch.utils.data.Dataset,
                  epochs: int,
                  criterion_validation: torch.nn.Module = None,
                  ):

        self.model = model
        self.criterion = criterion
        self.optimizer = optimizer
        self.training_DataLoader = training_DataLoader
        self.validation_DataLoader = validation_DataLoader
        self.device = device
        self.epochs = epochs

        if criterion_validation is None:
            self.criterion_validation = criterion
        else:
            self.criterion_validation = criterion_validation

    def run_trainer(self):
        """Training for all epochs. Returns training history."""

        # Dictionary to save training history
        history = {
            "train_loss": [],
            "val_loss": [],
            "macro_precision": [],
            "macro_recall": []
        }

        # Train and validate with loading bar
        for epoch in tqdm(range(1, self.epochs+1)):
            # Training
            self.model.train()
            train_losses = []

```

```

for batch in self.training_DataLoader:
    x, y = batch
    input, target = x.to(self.device), y.to(self.device)
    self.optimizer.zero_grad()
    out = self.model(input)
    loss = self.criterion(out, target)

    loss_value = loss.item()
    train_losses.append(loss_value)

    loss.backward()
    self.optimizer.step()

# Validation
self.model.eval()

valid_losses = []
all_true = []
all_pred = []

with torch.no_grad():
    for batch in self.validation_DataLoader:
        x, y = batch
        inputs = x.to(self.device)
        targets = y.to(self.device)

        out = self.model(inputs)
        loss = self.criterion_validation(out, targets)

        valid_losses.append(loss.item())

        preds = out.argmax(dim=1)

        all_true.append(targets.cpu())
        all_pred.append(preds.cpu())

macro_precision, macro_recall =
compute_macro_metrics(all_true, all_pred, num_classes=8)
avg_train_loss = np.mean(train_losses)
avg_val_loss = np.mean(valid_losses)

history["train_loss"].append(avg_train_loss)
history["val_loss"].append(avg_val_loss)
history["macro_precision"].append(macro_precision)
history["macro_recall"].append(macro_recall)

print(
    f'EPOCH:
{epoch:0>{len(str(self.epochs))}}/{self.epochs}',
    end=' '

```

```

    )
    print(f'LOSS: {avg_train_loss:.4f}', end=' ')
    print(f'VAL-LOSS: {avg_val_loss:.4f}', end=' ')
    print(f'MACRO-PRECISION: {macro_precision:.4f}', end=' ')
    print(f'MACRO-RECALL: {macro_recall:.4f}')

    return history

# Create dataloaders
train_loader3 = DataLoader(train_dataset, batch_size=32, shuffle=True)
test_loader3 = DataLoader(test_dataset, batch_size=32, shuffle=False)

## Linear probing via training
linprob_model = LinearProbeClassifier(num_classes=8).to(device)

# Same as Task 1
criterion3 = nn.CrossEntropyLoss()
optimizer3 = torch.optim.AdamW(linprob_model.parameters(), lr=1e-3,
weight_decay=1e-4)

# 10 epochs max for the linear head training
# Can use fewer to avoid overfitting
trainer = Trainer_task3(
    model=linprob_model,
    device=device,
    criterion=criterion3,
    optimizer=optimizer3,
    training_DataLoader=train_loader3,
    validation_DataLoader=test_loader3,
    epochs=10
)

history_lin = trainer.run_trainer()

# torch.save(linprob_model.state_dict(), "task_3_linear_probe_v3.pth")

{"model_id": "b717e00dece048238ec82ce20e12da12", "version_major": 2, "version_minor": 0}

[transformers] SwinModel LOAD REPORT from: microsoft/swin-large-patch4-window7-224
Key | Status | |
-----+-----+--
classifier.bias | UNEXPECTED | |
classifier.weight | UNEXPECTED | |

Notes:
- UNEXPECTED: can be ignored when loading from different task/architecture; not ok if you expect identical arch.
10%|██████████
| 1/10 [01:20<12:08, 80.95s/it]

```

EPOCH: 01/10 LOSS: 1.1758 VAL-LOSS: 0.8533 MACRO-PRECISION: 0.7341
MACRO-RECALL: 0.7292

20%|████████████████████
| 2/10 [02:41<10:46, 80.86s/it]

EPOCH: 02/10 LOSS: 0.7250 VAL-LOSS: 0.7636 MACRO-PRECISION: 0.7463
MACRO-RECALL: 0.7417

30%|████████████████████
| 3/10 [04:02<09:25, 80.81s/it]

EPOCH: 03/10 LOSS: 0.6136 VAL-LOSS: 0.7058 MACRO-PRECISION: 0.7517
MACRO-RECALL: 0.7500

40%|████████████████████
| 4/10 [05:23<08:05, 80.95s/it]

EPOCH: 04/10 LOSS: 0.5382 VAL-LOSS: 0.6917 MACRO-PRECISION: 0.7701
MACRO-RECALL: 0.7625

50%|████████████████████
| 5/10 [06:48<06:51, 82.30s/it]

EPOCH: 05/10 LOSS: 0.4898 VAL-LOSS: 0.6871 MACRO-PRECISION: 0.7594
MACRO-RECALL: 0.7583

60%|████████████████████
| 6/10 [08:11<05:30, 82.66s/it]

EPOCH: 06/10 LOSS: 0.4529 VAL-LOSS: 0.6839 MACRO-PRECISION: 0.7469
MACRO-RECALL: 0.7417

70%|████████████████████
| 7/10 [09:34<04:08, 82.68s/it]

EPOCH: 07/10 LOSS: 0.4175 VAL-LOSS: 0.6675 MACRO-PRECISION: 0.7579
MACRO-RECALL: 0.7583

80%|████████████████████
| 8/10 [10:56<02:44, 82.37s/it]

EPOCH: 08/10 LOSS: 0.3887 VAL-LOSS: 0.6840 MACRO-PRECISION: 0.7557
MACRO-RECALL: 0.7500

90%|████████████████████
| 9/10 [12:17<01:22, 82.10s/it]

EPOCH: 09/10 LOSS: 0.3630 VAL-LOSS: 0.6765 MACRO-PRECISION: 0.7426
MACRO-RECALL: 0.7417

100%|

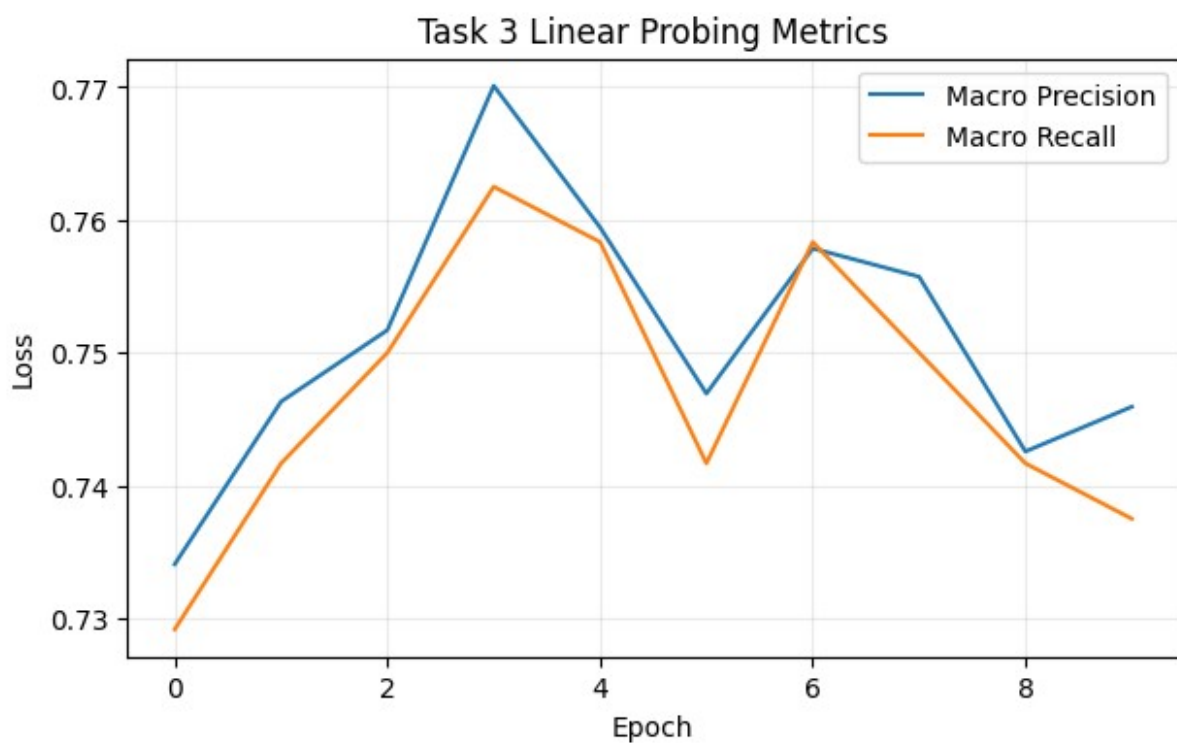
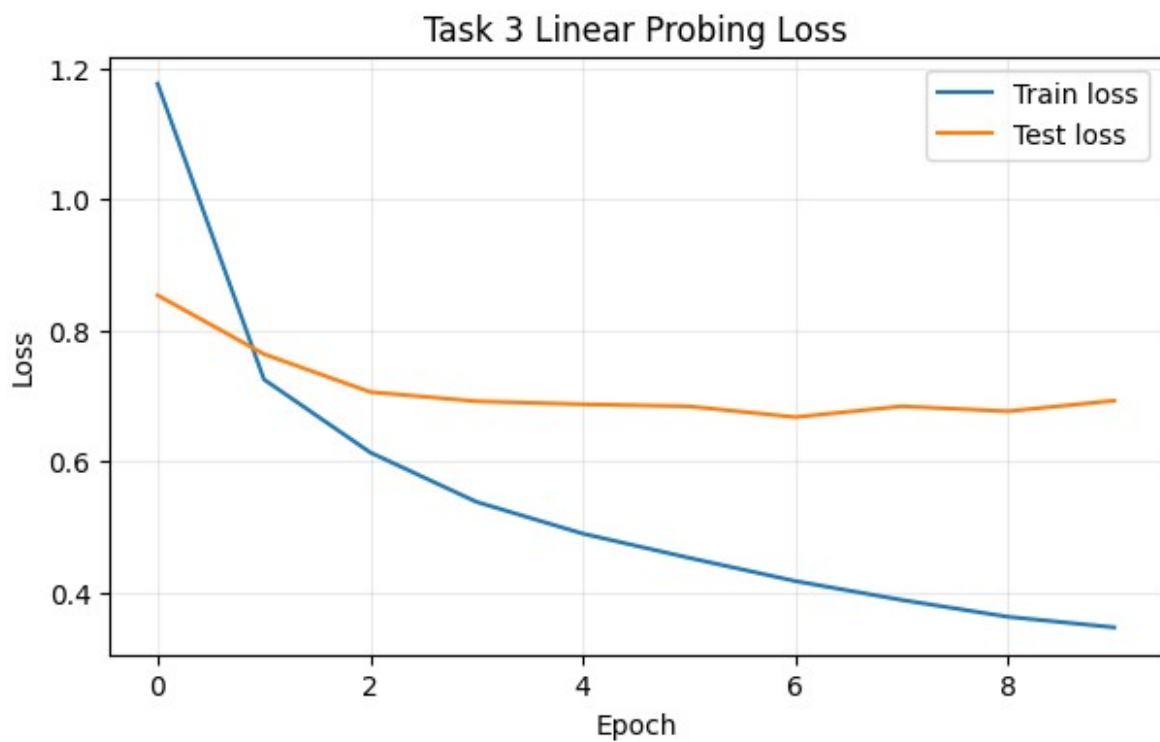
| 10/10 [13:39<00:00, 81.92s/it]

EPOCH: 10/10 LOSS: 0.3469 VAL-LOSS: 0.6926 MACRO-PRECISION: 0.7459
MACRO-RECALL: 0.7375

Plotting to check overfitting

```
plt.figure(figsize=(7, 4))
plt.plot(history_lin["train_loss"], label="Train loss")
plt.plot(history_lin["val_loss"], label="Test loss")
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.title("Task 3 Linear Probing Loss")
plt.legend()
plt.grid(alpha=0.25)
plt.show()

plt.figure(figsize=(7, 4))
plt.plot(history_lin["macro_precision"], label="Macro Precision")
plt.plot(history_lin["macro_recall"], label="Macro Recall")
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.title("Task 3 Linear Probing Metrics")
plt.legend()
plt.grid(alpha=0.25)
plt.show()
```



```
## kNN classification on backbone  
# Get linear probing backbone to save memory
```



```

knn_model = linprob_model.backbone
knn_model.eval()

SwinModel(
  (embeddings): SwinEmbeddings(
    (patch_embeddings): SwinPatchEmbeddings(
      (projection): Conv2d(3, 192, kernel_size=(4, 4), stride=(4, 4))
    )
    (norm): LayerNorm((192,)), eps=1e-05, elementwise_affine=True)
    (dropout): Dropout(p=0.0, inplace=False)
  )
  (encoder): SwinEncoder(
    (layers): ModuleList(
      (0): SwinStage(
        (blocks): ModuleList(
          (0): SwinLayer(
            (attention): SwinAttention(
              (q_proj): Linear(in_features=192, out_features=192,
bias=True)
              (k_proj): Linear(in_features=192, out_features=192,
bias=True)
              (v_proj): Linear(in_features=192, out_features=192,
bias=True)
              (o_proj): Linear(in_features=192, out_features=192,
bias=True)
              (relative_position_bias): SwinRelativePositionBias()
            )
            (layernorm_before): LayerNorm((192,)), eps=1e-05,
elementwise_affine=True)
            (layernorm_after): LayerNorm((192,)), eps=1e-05,
elementwise_affine=True)
            (mlp): SwinMLP(
              (activation_fn): GELUActivation()
              (fc1): Linear(in_features=192, out_features=768,
bias=True)
              (fc2): Linear(in_features=768, out_features=192,
bias=True)
            )
            (dropout): Dropout(p=0.0, inplace=False)
            (drop_path): Identity()
          )
          (1): SwinLayer(
            (attention): SwinAttention(
              (q_proj): Linear(in_features=192, out_features=192,
bias=True)
              (k_proj): Linear(in_features=192, out_features=192,
bias=True)
              (v_proj): Linear(in_features=192, out_features=192,
bias=True)
              (o_proj): Linear(in_features=192, out_features=192,

```

```

bias=True)
    (relative_position_bias): SwinRelativePositionBias()
    )
    (layernorm_before): LayerNorm((192,), eps=1e-05,
elementwise_affine=True)
    (layernorm_after): LayerNorm((192,), eps=1e-05,
elementwise_affine=True)
    (mlp): SwinMLP(
        (activation_fn): GELUActivation()
        (fc1): Linear(in_features=192, out_features=768,
bias=True)
        (fc2): Linear(in_features=768, out_features=192,
bias=True)
    )
    (dropout): Dropout(p=0.0, inplace=False)
    (drop_path): SwinDropPath(p=0.004347826086956522)
    )
    )
    (downsample): SwinPatchMerging(
        (reduction): Linear(in_features=768, out_features=384,
bias=False)
        (norm): LayerNorm((768,), eps=1e-05,
elementwise_affine=True)
    )
    )
    (1): SwinStage(
        (blocks): ModuleList(
            (0): SwinLayer(
                (attention): SwinAttention(
                    (q_proj): Linear(in_features=384, out_features=384,
bias=True)
                    (k_proj): Linear(in_features=384, out_features=384,
bias=True)
                    (v_proj): Linear(in_features=384, out_features=384,
bias=True)
                    (o_proj): Linear(in_features=384, out_features=384,
bias=True)
                    (relative_position_bias): SwinRelativePositionBias()
                )
                (layernorm_before): LayerNorm((384,), eps=1e-05,
elementwise_affine=True)
                (layernorm_after): LayerNorm((384,), eps=1e-05,
elementwise_affine=True)
                (mlp): SwinMLP(
                    (activation_fn): GELUActivation()
                    (fc1): Linear(in_features=384, out_features=1536,
bias=True)
                    (fc2): Linear(in_features=1536, out_features=384,
bias=True)
                )
            )
        )
    )

```

```

        (dropout): Dropout(p=0.0, inplace=False)
        (drop_path): SwinDropPath(p=0.008695652173913044)
    )
    (1): SwinLayer(
      (attention): SwinAttention(
        (q_proj): Linear(in_features=384, out_features=384,
bias=True)
        (k_proj): Linear(in_features=384, out_features=384,
bias=True)
        (v_proj): Linear(in_features=384, out_features=384,
bias=True)
        (o_proj): Linear(in_features=384, out_features=384,
bias=True)
        (relative_position_bias): SwinRelativePositionBias()
      )
      (layernorm_before): LayerNorm((384,), eps=1e-05,
elementwise_affine=True)
      (layernorm_after): LayerNorm((384,), eps=1e-05,
elementwise_affine=True)
      (mlp): SwinMLP(
        (activation_fn): GELUActivation()
        (fc1): Linear(in_features=384, out_features=1536,
bias=True)
        (fc2): Linear(in_features=1536, out_features=384,
bias=True)
      )
      (dropout): Dropout(p=0.0, inplace=False)
      (drop_path): SwinDropPath(p=0.013043478260869568)
    )
  )
  (downsample): SwinPatchMerging(
    (reduction): Linear(in_features=1536, out_features=768,
bias=False)
    (norm): LayerNorm((1536,), eps=1e-05,
elementwise_affine=True)
  )
)
(2): SwinStage(
  (blocks): ModuleList(
    (0): SwinLayer(
      (attention): SwinAttention(
        (q_proj): Linear(in_features=768, out_features=768,
bias=True)
        (k_proj): Linear(in_features=768, out_features=768,
bias=True)
        (v_proj): Linear(in_features=768, out_features=768,
bias=True)
        (o_proj): Linear(in_features=768, out_features=768,
bias=True)
        (relative_position_bias): SwinRelativePositionBias()

```

```

        )
        (layernorm_before): LayerNorm((768,), eps=1e-05,
elementwise_affine=True)
        (layernorm_after): LayerNorm((768,), eps=1e-05,
elementwise_affine=True)
        (mlp): SwinMLP(
            (activation_fn): GELUActivation()
            (fc1): Linear(in_features=768, out_features=3072,
bias=True)
            (fc2): Linear(in_features=3072, out_features=768,
bias=True)
        )
        (dropout): Dropout(p=0.0, inplace=False)
        (drop_path): SwinDropPath(p=0.017391304347826087)
    )
    (1): SwinLayer(
        (attention): SwinAttention(
            (q_proj): Linear(in_features=768, out_features=768,
bias=True)
            (k_proj): Linear(in_features=768, out_features=768,
bias=True)
            (v_proj): Linear(in_features=768, out_features=768,
bias=True)
            (o_proj): Linear(in_features=768, out_features=768,
bias=True)
            (relative_position_bias): SwinRelativePositionBias()
        )
        (layernorm_before): LayerNorm((768,), eps=1e-05,
elementwise_affine=True)
        (layernorm_after): LayerNorm((768,), eps=1e-05,
elementwise_affine=True)
        (mlp): SwinMLP(
            (activation_fn): GELUActivation()
            (fc1): Linear(in_features=768, out_features=3072,
bias=True)
            (fc2): Linear(in_features=3072, out_features=768,
bias=True)
        )
        (dropout): Dropout(p=0.0, inplace=False)
        (drop_path): SwinDropPath(p=0.021739130434782608)
    )
    (2): SwinLayer(
        (attention): SwinAttention(
            (q_proj): Linear(in_features=768, out_features=768,
bias=True)
            (k_proj): Linear(in_features=768, out_features=768,
bias=True)
            (v_proj): Linear(in_features=768, out_features=768,
bias=True)
            (o_proj): Linear(in_features=768, out_features=768,

```

```

bias=True)
    (relative_position_bias): SwinRelativePositionBias()
    )
    (layernorm_before): LayerNorm((768,), eps=1e-05,
elementwise_affine=True)
    (layernorm_after): LayerNorm((768,), eps=1e-05,
elementwise_affine=True)
    (mlp): SwinMLP(
        (activation_fn): GELUActivation()
        (fc1): Linear(in_features=768, out_features=3072,
bias=True)
        (fc2): Linear(in_features=3072, out_features=768,
bias=True)
    )
    (dropout): Dropout(p=0.0, inplace=False)
    (drop_path): SwinDropPath(p=0.026086956521739136)
    )
    (3): SwinLayer(
        (attention): SwinAttention(
            (q_proj): Linear(in_features=768, out_features=768,
bias=True)
            (k_proj): Linear(in_features=768, out_features=768,
bias=True)
            (v_proj): Linear(in_features=768, out_features=768,
bias=True)
            (o_proj): Linear(in_features=768, out_features=768,
bias=True)
            (relative_position_bias): SwinRelativePositionBias()
        )
        (layernorm_before): LayerNorm((768,), eps=1e-05,
elementwise_affine=True)
        (layernorm_after): LayerNorm((768,), eps=1e-05,
elementwise_affine=True)
        (mlp): SwinMLP(
            (activation_fn): GELUActivation()
            (fc1): Linear(in_features=768, out_features=3072,
bias=True)
            (fc2): Linear(in_features=3072, out_features=768,
bias=True)
        )
        (dropout): Dropout(p=0.0, inplace=False)
        (drop_path): SwinDropPath(p=0.030434782608695653)
    )
    (4): SwinLayer(
        (attention): SwinAttention(
            (q_proj): Linear(in_features=768, out_features=768,
bias=True)
            (k_proj): Linear(in_features=768, out_features=768,
bias=True)
            (v_proj): Linear(in_features=768, out_features=768,

```

```

bias=True)
        (o_proj): Linear(in_features=768, out_features=768,
bias=True)
        (relative_position_bias): SwinRelativePositionBias()
    )
    (layernorm_before): LayerNorm((768,), eps=1e-05,
elementwise_affine=True)
    (layernorm_after): LayerNorm((768,), eps=1e-05,
elementwise_affine=True)
    (mlp): SwinMLP(
        (activation_fn): GELUActivation()
        (fc1): Linear(in_features=768, out_features=3072,
bias=True)
        (fc2): Linear(in_features=3072, out_features=768,
bias=True)
    )
    (dropout): Dropout(p=0.0, inplace=False)
    (drop_path): SwinDropPath(p=0.034782608695652174)
)
(5): SwinLayer(
    (attention): SwinAttention(
        (q_proj): Linear(in_features=768, out_features=768,
bias=True)
        (k_proj): Linear(in_features=768, out_features=768,
bias=True)
        (v_proj): Linear(in_features=768, out_features=768,
bias=True)
        (o_proj): Linear(in_features=768, out_features=768,
bias=True)
        (relative_position_bias): SwinRelativePositionBias()
    )
    (layernorm_before): LayerNorm((768,), eps=1e-05,
elementwise_affine=True)
    (layernorm_after): LayerNorm((768,), eps=1e-05,
elementwise_affine=True)
    (mlp): SwinMLP(
        (activation_fn): GELUActivation()
        (fc1): Linear(in_features=768, out_features=3072,
bias=True)
        (fc2): Linear(in_features=3072, out_features=768,
bias=True)
    )
    (dropout): Dropout(p=0.0, inplace=False)
    (drop_path): SwinDropPath(p=0.0391304347826087)
)
(6): SwinLayer(
    (attention): SwinAttention(
        (q_proj): Linear(in_features=768, out_features=768,
bias=True)
        (k_proj): Linear(in_features=768, out_features=768,

```

```

bias=True)
    (v_proj): Linear(in_features=768, out_features=768,
bias=True)
    (o_proj): Linear(in_features=768, out_features=768,
bias=True)
    (relative_position_bias): SwinRelativePositionBias()
    )
    (layernorm_before): LayerNorm((768,), eps=1e-05,
elementwise_affine=True)
    (layernorm_after): LayerNorm((768,), eps=1e-05,
elementwise_affine=True)
    (mlp): SwinMLP(
        (activation_fn): GELUActivation()
        (fc1): Linear(in_features=768, out_features=3072,
bias=True)
        (fc2): Linear(in_features=3072, out_features=768,
bias=True)
    )
    (dropout): Dropout(p=0.0, inplace=False)
    (drop_path): SwinDropPath(p=0.043478260869565216)
    )
    (7): SwinLayer(
        (attention): SwinAttention(
            (q_proj): Linear(in_features=768, out_features=768,
bias=True)
            (k_proj): Linear(in_features=768, out_features=768,
bias=True)
            (v_proj): Linear(in_features=768, out_features=768,
bias=True)
            (o_proj): Linear(in_features=768, out_features=768,
bias=True)
            (relative_position_bias): SwinRelativePositionBias()
        )
        (layernorm_before): LayerNorm((768,), eps=1e-05,
elementwise_affine=True)
        (layernorm_after): LayerNorm((768,), eps=1e-05,
elementwise_affine=True)
        (mlp): SwinMLP(
            (activation_fn): GELUActivation()
            (fc1): Linear(in_features=768, out_features=3072,
bias=True)
            (fc2): Linear(in_features=3072, out_features=768,
bias=True)
        )
        (dropout): Dropout(p=0.0, inplace=False)
        (drop_path): SwinDropPath(p=0.04782608695652174)
    )
    (8): SwinLayer(
        (attention): SwinAttention(
            (q_proj): Linear(in_features=768, out_features=768,

```

```

bias=True)
    (k_proj): Linear(in_features=768, out_features=768,
bias=True)
    (v_proj): Linear(in_features=768, out_features=768,
bias=True)
    (o_proj): Linear(in_features=768, out_features=768,
bias=True)
    (relative_position_bias): SwinRelativePositionBias()
    )
    (layernorm_before): LayerNorm((768,), eps=1e-05,
elementwise_affine=True)
    (layernorm_after): LayerNorm((768,), eps=1e-05,
elementwise_affine=True)
    (mlp): SwinMLP(
        (activation_fn): GELUActivation()
        (fc1): Linear(in_features=768, out_features=3072,
bias=True)
        (fc2): Linear(in_features=3072, out_features=768,
bias=True)
    )
    (dropout): Dropout(p=0.0, inplace=False)
    (drop_path): SwinDropPath(p=0.05217391304347827)
    )
(9): SwinLayer(
    (attention): SwinAttention(
        (q_proj): Linear(in_features=768, out_features=768,
bias=True)
        (k_proj): Linear(in_features=768, out_features=768,
bias=True)
        (v_proj): Linear(in_features=768, out_features=768,
bias=True)
        (o_proj): Linear(in_features=768, out_features=768,
bias=True)
        (relative_position_bias): SwinRelativePositionBias()
    )
    (layernorm_before): LayerNorm((768,), eps=1e-05,
elementwise_affine=True)
    (layernorm_after): LayerNorm((768,), eps=1e-05,
elementwise_affine=True)
    (mlp): SwinMLP(
        (activation_fn): GELUActivation()
        (fc1): Linear(in_features=768, out_features=3072,
bias=True)
        (fc2): Linear(in_features=3072, out_features=768,
bias=True)
    )
    (dropout): Dropout(p=0.0, inplace=False)
    (drop_path): SwinDropPath(p=0.05652173913043478)
    )
(10): SwinLayer(

```



```

        (attention): SwinAttention(
          (q_proj): Linear(in_features=768, out_features=768,
bias=True)
          (k_proj): Linear(in_features=768, out_features=768,
bias=True)
          (v_proj): Linear(in_features=768, out_features=768,
bias=True)
          (o_proj): Linear(in_features=768, out_features=768,
bias=True)
          (relative_position_bias): SwinRelativePositionBias()
        )
        (layernorm_before): LayerNorm((768,), eps=1e-05,
elementwise_affine=True)
        (layernorm_after): LayerNorm((768,), eps=1e-05,
elementwise_affine=True)
        (mlp): SwinMLP(
          (activation_fn): GELUActivation()
          (fc1): Linear(in_features=768, out_features=3072,
bias=True)
          (fc2): Linear(in_features=3072, out_features=768,
bias=True)
        )
        (dropout): Dropout(p=0.0, inplace=False)
        (drop_path): SwinDropPath(p=0.06086956521739131)
      )
    (11): SwinLayer(
      (attention): SwinAttention(
        (q_proj): Linear(in_features=768, out_features=768,
bias=True)
        (k_proj): Linear(in_features=768, out_features=768,
bias=True)
        (v_proj): Linear(in_features=768, out_features=768,
bias=True)
        (o_proj): Linear(in_features=768, out_features=768,
bias=True)
        (relative_position_bias): SwinRelativePositionBias()
      )
      (layernorm_before): LayerNorm((768,), eps=1e-05,
elementwise_affine=True)
      (layernorm_after): LayerNorm((768,), eps=1e-05,
elementwise_affine=True)
      (mlp): SwinMLP(
        (activation_fn): GELUActivation()
        (fc1): Linear(in_features=768, out_features=3072,
bias=True)
        (fc2): Linear(in_features=3072, out_features=768,
bias=True)
      )
      (dropout): Dropout(p=0.0, inplace=False)
      (drop_path): SwinDropPath(p=0.06521739130434782)
    )
  )
)

```

```

    )
    (12): SwinLayer(
      (attention): SwinAttention(
        (q_proj): Linear(in_features=768, out_features=768,
bias=True)
        (k_proj): Linear(in_features=768, out_features=768,
bias=True)
        (v_proj): Linear(in_features=768, out_features=768,
bias=True)
        (o_proj): Linear(in_features=768, out_features=768,
bias=True)
        (relative_position_bias): SwinRelativePositionBias()
      )
      (layernorm_before): LayerNorm((768,), eps=1e-05,
elementwise_affine=True)
      (layernorm_after): LayerNorm((768,), eps=1e-05,
elementwise_affine=True)
      (mlp): SwinMLP(
        (activation_fn): GELUActivation()
        (fc1): Linear(in_features=768, out_features=3072,
bias=True)
        (fc2): Linear(in_features=3072, out_features=768,
bias=True)
      )
      (dropout): Dropout(p=0.0, inplace=False)
      (drop_path): SwinDropPath(p=0.06956521739130435)
    )
    (13): SwinLayer(
      (attention): SwinAttention(
        (q_proj): Linear(in_features=768, out_features=768,
bias=True)
        (k_proj): Linear(in_features=768, out_features=768,
bias=True)
        (v_proj): Linear(in_features=768, out_features=768,
bias=True)
        (o_proj): Linear(in_features=768, out_features=768,
bias=True)
        (relative_position_bias): SwinRelativePositionBias()
      )
      (layernorm_before): LayerNorm((768,), eps=1e-05,
elementwise_affine=True)
      (layernorm_after): LayerNorm((768,), eps=1e-05,
elementwise_affine=True)
      (mlp): SwinMLP(
        (activation_fn): GELUActivation()
        (fc1): Linear(in_features=768, out_features=3072,
bias=True)
        (fc2): Linear(in_features=3072, out_features=768,
bias=True)
      )
    )

```

```

        (dropout): Dropout(p=0.0, inplace=False)
        (drop_path): SwinDropPath(p=0.07391304347826087)
    )
    (14): SwinLayer(
      (attention): SwinAttention(
        (q_proj): Linear(in_features=768, out_features=768,
bias=True)
        (k_proj): Linear(in_features=768, out_features=768,
bias=True)
        (v_proj): Linear(in_features=768, out_features=768,
bias=True)
        (o_proj): Linear(in_features=768, out_features=768,
bias=True)
        (relative_position_bias): SwinRelativePositionBias()
      )
      (layernorm_before): LayerNorm((768,), eps=1e-05,
elementwise_affine=True)
      (layernorm_after): LayerNorm((768,), eps=1e-05,
elementwise_affine=True)
      (mlp): SwinMLP(
        (activation_fn): GELUActivation()
        (fc1): Linear(in_features=768, out_features=3072,
bias=True)
        (fc2): Linear(in_features=3072, out_features=768,
bias=True)
      )
      (dropout): Dropout(p=0.0, inplace=False)
      (drop_path): SwinDropPath(p=0.0782608695652174)
    )
    (15): SwinLayer(
      (attention): SwinAttention(
        (q_proj): Linear(in_features=768, out_features=768,
bias=True)
        (k_proj): Linear(in_features=768, out_features=768,
bias=True)
        (v_proj): Linear(in_features=768, out_features=768,
bias=True)
        (o_proj): Linear(in_features=768, out_features=768,
bias=True)
        (relative_position_bias): SwinRelativePositionBias()
      )
      (layernorm_before): LayerNorm((768,), eps=1e-05,
elementwise_affine=True)
      (layernorm_after): LayerNorm((768,), eps=1e-05,
elementwise_affine=True)
      (mlp): SwinMLP(
        (activation_fn): GELUActivation()
        (fc1): Linear(in_features=768, out_features=3072,
bias=True)
        (fc2): Linear(in_features=3072, out_features=768,

```

```

bias=True)
    )
    (dropout): Dropout(p=0.0, inplace=False)
    (drop_path): SwinDropPath(p=0.08260869565217392)
    )
    (16): SwinLayer(
      (attention): SwinAttention(
        (q_proj): Linear(in_features=768, out_features=768,
bias=True)
        (k_proj): Linear(in_features=768, out_features=768,
bias=True)
        (v_proj): Linear(in_features=768, out_features=768,
bias=True)
        (o_proj): Linear(in_features=768, out_features=768,
bias=True)
        (relative_position_bias): SwinRelativePositionBias()
      )
      (layernorm_before): LayerNorm((768,), eps=1e-05,
elementwise_affine=True)
      (layernorm_after): LayerNorm((768,), eps=1e-05,
elementwise_affine=True)
      (mlp): SwinMLP(
        (activation_fn): GELUActivation()
        (fc1): Linear(in_features=768, out_features=3072,
bias=True)
        (fc2): Linear(in_features=3072, out_features=768,
bias=True)
      )
      (dropout): Dropout(p=0.0, inplace=False)
      (drop_path): SwinDropPath(p=0.08695652173913043)
    )
    (17): SwinLayer(
      (attention): SwinAttention(
        (q_proj): Linear(in_features=768, out_features=768,
bias=True)
        (k_proj): Linear(in_features=768, out_features=768,
bias=True)
        (v_proj): Linear(in_features=768, out_features=768,
bias=True)
        (o_proj): Linear(in_features=768, out_features=768,
bias=True)
        (relative_position_bias): SwinRelativePositionBias()
      )
      (layernorm_before): LayerNorm((768,), eps=1e-05,
elementwise_affine=True)
      (layernorm_after): LayerNorm((768,), eps=1e-05,
elementwise_affine=True)
      (mlp): SwinMLP(
        (activation_fn): GELUActivation()

```

```

        (fc1): Linear(in_features=768, out_features=3072,
bias=True)
        (fc2): Linear(in_features=3072, out_features=768,
bias=True)
    )
    (dropout): Dropout(p=0.0, inplace=False)
    (drop_path): SwinDropPath(p=0.09130434782608696)
)
)
(downsample): SwinPatchMerging(
  (reduction): Linear(in_features=3072, out_features=1536,
bias=False)
  (norm): LayerNorm((3072,), eps=1e-05,
elementwise_affine=True)
)
)
(3): SwinStage(
  (blocks): ModuleList(
    (0): SwinLayer(
      (attention): SwinAttention(
        (q_proj): Linear(in_features=1536, out_features=1536,
bias=True)
        (k_proj): Linear(in_features=1536, out_features=1536,
bias=True)
        (v_proj): Linear(in_features=1536, out_features=1536,
bias=True)
        (o_proj): Linear(in_features=1536, out_features=1536,
bias=True)
        (relative_position_bias): SwinRelativePositionBias()
      )
      (layernorm_before): LayerNorm((1536,), eps=1e-05,
elementwise_affine=True)
      (layernorm_after): LayerNorm((1536,), eps=1e-05,
elementwise_affine=True)
      (mlp): SwinMLP(
        (activation_fn): GELUActivation()
        (fc1): Linear(in_features=1536, out_features=6144,
bias=True)
        (fc2): Linear(in_features=6144, out_features=1536,
bias=True)
      )
      (dropout): Dropout(p=0.0, inplace=False)
      (drop_path): SwinDropPath(p=0.09565217391304348)
    )
    (1): SwinLayer(
      (attention): SwinAttention(
        (q_proj): Linear(in_features=1536, out_features=1536,
bias=True)
        (k_proj): Linear(in_features=1536, out_features=1536,

```



```

embeddings = torch.cat(embeddings, dim=0).cpu()
labels = torch.cat(labels, dim=0).cpu()

return embeddings, labels

## This code is taken from this website:
https://www.datasciencebase.com/supervised-ml/algorithms/k-nearest-
neighbors/pytorch-example/
# Define the function to calculate Euclidean distance
def euclidean_distance(X_train, X_test_point):
    # Subtract the test point from each training point and compute the
squared distances
    distances = torch.sum((X_train - X_test_point) ** 2, axis=1)
    return torch.sqrt(distances)

# Define the KNN classifier function
def knn(X_train, y_train, X_test, K):
    y_pred = []

    for X_test_point in X_test:
        # Compute distances from the test point to all training points
        distances = euclidean_distance(X_train, X_test_point)

        # Get the indices of the K nearest neighbors
        nearest_neighbors = torch.argsort(distances)[:K]

        # Get the labels of the K nearest neighbors
        nearest_labels = y_train[nearest_neighbors]

        # Perform a majority vote to predict the label
        predicted_label = torch.mode(nearest_labels).values.item()
        y_pred.append(predicted_label)

    return torch.tensor(y_pred)

# Get embeddings
train_embeddings, train_labels = extract_embeddings(knn_model,
train_loader3, device)
test_embeddings, test_labels = extract_embeddings(knn_model,
test_loader3, device)

# Run kNN and get precision and recall metrics
predictions = knn(train_embeddings, train_labels, test_embeddings,
K=5)

precision_knn = precision_score(test_labels, predictions,
average="macro")
recall_knn = recall_score(test_labels, predictions, average="macro")

```

```
print(f"Final kNN macro precision: \t{precision_knn}\nFinal kNN macro  
recall: \t{recall_knn}")
```

```
Final kNN macro precision: 0.6347001190751191  
Final kNN macro recall: 0.6291666666666667
```

```
## Final visualization
```

```
# Task 2 final recall and precision
```

```
# TODO: get results for barplot, has placeholders for plotting
```

```
final_recall_task2 = macro_recall_clean
```

```
final_prec_task = macro_prec_clean
```

```
# Linear probe
```

```
final_recall_lp = history_lin["macro_recall"][-1]
```

```
final_prec_lp = history_lin["macro_precision"][-1]
```

```
# kNN final results
```

```
final_recall_knn = recall_knn
```

```
final_prec_knn = precision_knn
```

```
# Organize data for plotting
```

```
methods = ["Task 2", "Linear Probe", "kNN"]
```

```
recall = [final_recall_task2, final_recall_lp, final_recall_knn]
```

```
precision = [final_prec_task, final_prec_lp, final_prec_knn]
```

```
# Bar chart with 6 bars, 3 methods * 2 metrics
```

```
x = np.arange(len(methods))
```

```
width = 0.35
```

```
plt.figure(figsize=(8, 5))
```

```
# Add bars with metric values per bar
```

```
bars1 = plt.bar(x - width/2, precision, width, label="Macro  
Precision")
```

```
bars2 = plt.bar(x + width/2, recall, width, label="Macro Recall")
```

```
ax = plt.gca()
```

```
ax.bar_label(bars1, fmt="%.2f", padding=3)
```

```
ax.bar_label(bars2, fmt="%.2f", padding=3)
```

```
# Task 2 minimum is 0.3 for both, so a line is added for distinction  
plt.axhline(0.3, linestyle="--", color="tab:red", label="Required  
minimum Task 2")
```

```
plt.xticks(x, methods)
```

```
plt.ylabel("Score")
```

```
plt.title("Performance Comparison Across Different Methods")
```

```
plt.ylim(0, 1.0)
```

```
plt.legend()
```

```
plt.savefig(
```



```
    "task3_comparison bar plot.png",  
    dpi=300,  
    bbox_inches="tight"  
)  
plt.show()
```

